

ՇԱՊԻԿ
«ԱԲ սերունդ» ծրագիր
Համակարգչային գիտություն
Python
Ծրագրավորման լեզվով

Դասագիրք
Տարբերակ 1.0
10-րդ դասարան

FAST logo, ԿԳՄՍ լոգո
2025թ., Երևան

Նախաբան

Այսօր ծրագրավորումն ընկալվում է ոչ միայն որպես գործնական հմտություն, այլև որպես համաշխարհային կրթական գրագիտության առանցքային բաղադրիչ: Python-ը, իր պարզ կառուցվածքով և ընթերցելիությամբ, համարվում է ամենամատչելի և բազմակողմանի ծրագրավորման լեզուներից մեկը՝ հնարավորություն ընձեռելով միջին և ավագ դասարանների սովորողներին վստահությամբ կատարել առաջին քայլերը ծրագրավորման աշխարհում:

ՀՀ հանրակրթության առարկայական չափորոշիչներով Python-ի հիմունքները ներառված են «Թվային գրագիտություն և համակարգչային գիտություն» առարկայի շրջանակում՝ սկսած 7-րդ դասարանից, իսկ «ԱԲ սերունդ» ծրագրում այն նախատեսված է որպես համակարգչային գիտության հիմնական լեզու 10-րդ և 11-րդ դասարաններում: Այնուամենայնիվ, հայերենում առկա ուսումնական նյութերը սահմանափակ են. բացատրությունները հաճախ չափազանց բարդ և խրթին են, ինչի պատճառով դժվարանում են հասկանալ թե՛ ուսուցիչները, թե՛ աշակերտները:

Սույն դասագիրքը մշակվել է հաշվի առնելով երկար տարիների Python-ի դասավանդման փորձը, ուսուցիչների և աշակերտների կողմից բարձրաձայնված կարիքները և միջազգային համակարգչային գիտության կրթական միտումները: Դասագիրքը նպատակ ունի պարզ ու հասկանալի բացատրություններով զարգացնել սովորողների կոդավորման և խնդիրներ լուծելու հմտությունները՝ տարականից մինչև բարդ մակարդակ: Առաջադրանքները խրախուսում են քննադատական մտածողություն, գործնական մոտեցում և ինքնուսուցում՝ ձևավորելով ամուր հիմք հետագա խորացված ուսման և կիրառությունների համար:

Python ուսումնական դասագրքի մշակող-փորձագետ՝

Գոռ Հովսեփյան, ԳՏՏՄ փորձագետ, Python լեզվի դասավանդող

Python ուսումնական դասագրքի վերանայող փորձագետներ՝

Միհրան Ավագյան, ԳՏՏՄ մանկավարժ-փորձագետ

Ավագ Սայան, Տեխնոկրթության փորձագետ

Երախտիքի խոսքեր.

Մենք մեր երախտագիտությունն ենք հայտնում «ԱԲ սերունդ» ծրագրի շուրջ ձևավորված մասնագիտական համայնքին՝ մշակվող նյութերը պարբերաբար վերանայելու և բարելավմանն ուղղված դիտարկումներ տրամադրելու համար: Դասագրքի այս տարբերակը գտնվում է փորձարկման փուլում:

«Սույն նյութի, ներառյալ դրա ամբողջ բովանդակության, ձևավորման, տեքստերի, պատկերների և ցանկացած այլ տարրի հեղինակային իրավունքը պատկանում է Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամին:

Սույն նյութը չի կարող ամբողջությամբ կամ մասնակիորեն վերարտադրվել, տարածվել, փոխանցվել, վերախմբագրվել, թարգմանվել կամ որևէ այլ ձևով օգտագործվել առանց Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամի նախնական գրավոր համաձայնության՝ անկախ օգտագործման նպատակից:

Սույն նյութի մեջ արտահայտված տեսակետները, վերլուծություններն ու մեկնաբանությունները հանդիսանում են հեղինակների մտավոր սեփականությունը:

© 2025, Հայաստանի գիտության և տեխնոլոգիաների հիմնադրամ Բոլոր իրավունքները պաշտպանված են:»

ԲՈՎԱՆԴԱԿՈՒԹՅՈՒՆ

Թեմա 1 ☐ Փոփոխականներ (Variables).....

Թեմա 2. Տողեր (String)

Թեմա 3՝ Ցիկլեր (Loops) For While.....

Թեմա 4՝ Պայմաններ (Conditions).....

Թեման 5՝ Set, Tuple (հավաքածուներ).....

Թեմա 6՝ Բառարան (Dictionary).....

Թեմա 7՝ Ֆունկցիաներ (Functions).....

Թեմա 8՝ Ֆայլեր (Files)

Թեմա 9՝ Խորացված Ֆունկցիաներ (Advanced Functions).....

Կարճ Ֆունկցիաներ.....

 Գրականություն և արտաքին հղումներ

Թեմա 1. Փոփոխականներ (Variables)

Ծրագրավորման մեջ մեզ հաճախ անհրաժեշտ է աշխատել տարբեր տիպի տվյալների հետ, որոնք անհրաժեշտ է պահպանել ծրագրի աշխատանքի ընթացքում՝ հետագայում օգտագործելու համար: Համակարգչի հիշողությունը կարելի է նմանեցնել բազմաբնակարան շենքի:

Ճենքում բնակվող ընտանիքներին կարելի է համեմատել տվյալների հետ, որոնցից յուրաքանչյուրն ունի իր համապատասխան բնակարանը: Ճենքերում (հատկապես՝ Սովետական Միության ժամանակ կառուցվածներում) հաճախ կարելի է տեսնել, որ բնակարանների դռներին փակցված է բնակվող ընտանիքի ազգանունը:

Փոփոխականը հենց այդ տվյալն է (այս դեպքում՝ ընտանիքը), որը գտնվում է համակարգչի հիշողության համապատասխան հատվածում (բազմաբնակարան շենքի համապատասխան բնակարանում), և նրան տարբերակելու համար օգտագործվում է անունը (այս դեպքում՝ բնակարանի դռանը փակցված ազգանունը):

Փոփոխականի տիպեր

Ծրագրավորման մեջ պահվող տվյալից և դրա տարբերակիչ անունից բացի, կարևոր է նաև դիտարկել տվյալների տեսակը: Ստորև ներկայացված աղյուսակում պարունակվում են տվյալների հիմնական տիպերը և դրանց բացատրությունները:

Տիպը	Բացատրություն	C++	Java	Python
int	Ամբողջ թվեր	int	int	int
float	Տասնորդական թվեր	float, double	float, double	float
bool	Տրամաբանական արժեք՝ արժեք True կամ False	bool	boolean	bool
char	1 սիմվոլ	char	char	Առկա չէ Կիրառվում է str (մեկ նիշ)
string	Տեքստ	std::string	String	str
list / array	Նույն տիպի տարրերի հաջորդականություն	array, vector	array, ArrayList	list
tuple	Տարբեր տիպի տվյալների անփոփոխելի հավաքածու	Առկա չէ	Առկա չէ	tuple
set	Չկրկնվող արժեքների հավաքածու	std::set	HashSet, TreeSet	set
dictionary / map	Բանալի-արժեք զույգեր (Օրինակ՝ բառարանն անգլերեն և դրան համարժեք հայերեն թարգմանությունը)	std::map	HashMap	dict
long	Երկար ամբողջ թիվ	long, long long	long	Առկա չէ Օգտագործվում է int
short	Կարճ ամբողջ թիվ	short	short	Առկա չէ Օգտագործվում է int
byte	Փոքր ամբողջ թիվ՝ 0-255 (կամ -128-127)	unsigned char/char	byte	Առկա չէ Օգտագործվում է int
object	Օբյեկտ՝ custom class կամ built-in	Ցանկացած class	Ցանկացած class	object

Փոփոխականի ստեղծում

Ծրագրավորման մեջ փոփոխական ստեղծելիս նախ հարկավոր է հասկանալ, թե ինչ տիպի (Տեսակի) տվյալ է հարկավոր պահել, այնուհետև, հասկանալ դրա սկզբնական արժեքը:

Վերոնշյալ հատվածում ստեղծել ենք `int` տեսակի փոփոխական, որի անունը `variableName` է, և դրան տրվել է սկզբնական արժեք՝ 5:

Python ծրագրավորման լեզվում փոփոխականի տիպը ստեղծելիս անհրաժեշտ չէ նշել այն, քանի որ որոշվում է ավտոմատ՝ ելնելով տրված սկզբնական արժեքից: Այս դեպքում, քանի որ մուտքային արժեքն ամբողջ թիվ է (5), ապա փոփոխականի տիպը կլինի `int`:

Կարևոր է հիշել

Փոփոխականի անվանումը կարող է լինել գրեթե ցանկացած տեսքի, սակայն պետք է հետևել որոշ հիմնական սահմանափակումների:

Կարող է՝

- Պարունակել լատինական տառեր՝ մեծատառ (A–Z) և փոքրատառ (a–z), կախված միջավայրից՝ կարող են օգտագործվել նաև Unicode տառեր (օրինակ՝ հայերեն), սակայն սա խորհուրդ չի տրվում:
- Պարունակել թվեր, բայց ոչ անունի սկզբում:
- Պարունակել հատուկ սիմվոլ՝ underscore (`_`), ցանկացած դիրքում:

Չի կարող՝

- Սկսվել թվով – թիվը կարող է հայտնվել միայն անվան միջնամասում կամ վերջում:

2. Օգտագործել արգելված սիմվոլներ, ինչպիսին են՝

`! @ # $ % ^ & * () - + = { } [] | \ : ; " ' < > , . ? / ~ ``

Կարևոր է նաև հիշել, որ Python-ում փոփոխականի անվանումները զգայուն են մեծատառի նկատմամբ: Այսինքն՝ Number և number փոփոխականները տարբեր փոփոխականներ են: Ինչպես նաև ցանկալի չէ, որ փոփոխականների անվանումները համընկնեն Python-ի բնակալի բառերի հետ, օրինակ՝ if, list, max և այլն: Սա կարող է առաջացնել սխալներ կամ սխալ մեկնաբանություններ ծրագրի մեջ:

Փոփոխականների պահումը համակարգչի հիշողության մեջ(Լրացուցիչ)

Համակարգչի հիշողության մեջ փոփոխականները պահվում են 2 տարբեր հատվածներում՝

1. Stack (յստեղ պահվում է մեր կողմից փոփոխականի տրված արժեքը և հիշողության տարածքը որ զբաղեցնում է այդ փոփոխականը):
2. Heap (Այստեղ պահվում է փոփոխականի արժեքը):

Երբ փորձում ենք հասնել փոփոխականի արժեքին, Python-ը նախ նայում է **Stack-ում պահված անունին (identifier)**: Այդ անունը **հղում է տալիս (reference)** մի հասցեի, որը գտնվում է **Heap-ում**, որտեղ պահվում է տվյալ արժեքը:

Սա կարելի է համեմատել բազմաբնակարան շենքի օրինակի հետ.

Stack-ում պահվում է գրանցամատյան՝ որտեղ նշված է միայն ընտանիքի անունը (օրինակ՝ «Պետրոսյան»):

Այդ գրանցման մեջ նշված է հասցե, որը տանում է դեպի տվյալ բնակարանը Heap-ում:

Երբ այցելում եք այդ հասցեով, գտնում եք ամբողջ ընտանիքը, այսինքն՝ արժեքը, որին հղվում էր անունը:

Անուն	Տիպ	Հասցե
Պետրոսյան	int	1407300. . .
variableName	int	2507300. . .

Ծրագրում ստեղծել ենք variableName անունով փոփոխական, որն իր մեջ պահում է ամբողջ թիվ՝ 5: Երբ այս հրամանն աշխատում է, **Stack-ում** ավելանում է նոր գրառում, որտեղ նշվում է փոփոխականի

- անունը,
- տիպը (օրինակ՝ int)
- և **հղումը (հասցեն)** դեպի **Heap**, որտեղ էլ հատկացվում է հիշողության հատված՝ հենց արժեքը պահելու համար, որն այս դեպքում կլինի 5 թիվը:

Երկրորդ տողում տպում ենք variableName փոփոխականի արժեքը:
 Այս գործողության ընթացքում Python-ը կատարում է հետևյալ քայլերը. սկզբում որոնում է variableName անունը **Stack-ում**: Գտնելուց հետո ստանում է **հասցեն**, որը հղում է դեպի **Heap**: Այդ հասցեով անցնում է Heap և այնտեղից **ստանում է պահված արժեքը**՝ այս դեպքում 5: Վերջում այդ արժեքը լռելյայն կերպով դրվում է variableName-ի փոխարեն և տպվում է:

Stack-ում պահվում է փոփոխականի տեսակը, որպեսզի համակարգիչը հասկանա, թե ինչ ծավալով տարածք է հարկավոր հատկացնել՝ տվյալ տիպի արժեք պահելու համար: Իդեալ, փոփոխականները, կախված իրենց տիպերից, ունեն նաև սահմաններ: Ստորև կարող եք տեսնել, թե որ փոփոխականն ինչպիսի արժեքներ է թույլատրում և ինչ չափ է զբաղեցնում:

Տիպ	Նկարագրություն	Թույլատրելի արժեքներ	Հիշողության ծավալ (մոտավոր)
int	Ամբողջ թիվ	Տեսականորեն սահմանափակված է միայն համակարգչի օպերատիվ հիշողությամբ: Օրինակ՝ -999999999999999... մինչև 999999999999999...	24 բայթ + կախված արժեքից
bool	Տրամաբանական արժեք	Միայն True կամ False	28 բայթ
float	Կոտորակային թիվ (64-bit IEEE 754)	Նվազագույն՝ $\sim -1.7976931348623157e+308$ Առավելագույն՝ $\sim 1.7976931348623157e+308$	24 բայթ
str	Տեքստային տիպ	Ցանկացած Unicode սիմվոլներ: Տեսականորեն սահմանափակված է միայն համակարգչի օպերատիվ հիշողությամբ:	49 բայթ + յուրաքանչյուր սիմվոլի համար

Այժմ դիտարկենք, թե ինչպես կարելի է ստեղծել տարբեր տիպի փոփոխականներ:

Ամբողջ թիվ պարունակող int տեսակի փոփոխականի ստեղծում:

Տասնորդական թիվ պարունակող float տեսակի փոփոխականի ստեղծում.

Տեքստ կամ 1 սիմվոլ պարունակող string (str) տեսակի փոփոխականի ստեղծում:

Տեքստային տիպերն ունեն մեկ առանձնահատկություն՝ ի տարբերություն int կամ float տիպերի, որտեղ արժեքն ուղղակի գրվում է: Այստեղ հարկավոր է տեքստը կամ սիմվոլը գրել անգլերեն չափերտների մեջ՝ " " կամ ` , որպեսզի ծրագիրը դա ընկալի որպես տեքստ:

Տրամաբանական տիպի (bool) փոփոխականների ստեղծում:

Տրամաբանական տիպի (bool) փոփոխականները կարող են ընդունել միայն երկու արժեք՝ դրանք են՝ True (ճիշտ) կամ False (Սխալ):

Գործողություններ փոփոխականների հետ

Փոփոխականների հետ կարելի է կատարել ցանկացած տիպի գործողություն, որը թույլատրված է կատարել այդ փոփոխականի արժեքի հետ: Դիտարկենք հենց մաթեմատիկական գործողությունները, որոնք կարելի է անել վերոնշյալ ցանկացած տիպի հետ (int, float, bool, str): Սակայն կախված փոփոխականի տիպից՝ գործողությունները կարող են կատարել տարբեր դեր:

Python – ում առկա մաթեմատիկական գործողությունները:

Օպերատոր	Նկարագրություն	Օրինակ	Արդյունք
+	Գումարում	3 + 2	5
-	Հանում	5 - 1	4
*	Բազմապատկում	4 * 2	8
/	Բաժանում (միշտ վերադարձնում է float)	7 / 2	3.5

//	Ամբողջ թվով բաժանում	7 // 2	3
%	Մնացորդ (մոդուլ)	7 % 2	1
**	Աստիճան բարձրացում (Exponent)	2 ** 3	8

Գումարում

Երկու int տիպի փոփոխականների գումարում:

Այս դեպքում ծրագիրը տպում է num1 և num2 փոփոխականների գումարը՝ 13: Սակայն այդ արժեքը ոչ մի տեղ չի պահվում, այլ միայն ցուցադրվում է:

Ստորև բերված օրինակում արժեքը պահվում է result փոփոխականում, որը նոր է ստեղծվում, այնուհետև, տպվում է result փոփոխականում առկա տվյալը: Այս դեպքում գումարման արդյունքը պահվում է, և հետագայում կարելի է գործողություն կատարել նաև արդյունքի

հետ:

Գումարման արդյունքում պարտադիր չէ ստեղծել նոր փոփոխական: Եթե հարկավոր է արժեքը պահել, կարելի է պահել հենց այն փոփոխականների մեջ, որոնց գումարը հաշվում ենք: Այս դեպքում առաջինը կատարվում է հավասարից աջ գրված գործողությունը $\text{num1} + \text{num2}$, որի արդյունքը՝ 13-ը, գրանցվում է num1 փոփոխականում: Այս նույն գործողությունը կարելի է անել նաև՝ օգտագործելով += օպերատորը, որը ներկայացված է ստորև:

Գումարման գործողությունը կարելի է նաև կատարել ոչ միայն փոփոխականների միջև, այլև նաև փոփոխականի և թվի միջև:

Հանում

Հանման գործողությունը Python-ում կատարում է նույն աշխատանքը, ինչ մաթեմատիկայում, և ունի կիրառման նույն հնարավորությունները, ինչ գումարումը, ինչպես նաև օգտագործվում է `-=` օպերատորը: Նշված օրինակում `num1`-ի արժեքից հանում է `num2`-ի արժեքը և հանում է

7, իսկ ստացված արժեքը (9 - ը) պահում է `num1` փոփոխականում:

Բազմապատկում և Աստիճան բարձրացում (Exponent)

Բազմապատկման գործողությունը python-ում նշվում է `*` սիմվոլի օգնությամբ և ունի կիրառման վերոնշյալ բոլոր ձևերը, այդ թվում՝ նաև `*=` տարբերակը:

Աստիճան բարձրացնելը կատարվում է 2 աստղանիշ նշանի օգնությամբ `**` և կարող է կիրառվել վերոնշյալ բոլոր տարբերակներով, այդ թվում՝ `**=` տարբերակով: Ստորև բերված օրինակում՝ ծրագիրը հաշվում է `num1`-ի խորանարդը և այն պահում է հենց `num1` փոփոխականում, ըստ այդմ՝ ծրագրի արդյունքը կլինի 27:

Բաժանում

Ծրագրավորման մեջ առկա է բաժանման 3 տարբերակ, որոնք են.

1. `/` -սա մաթեմաթիկայի սովորական տասնորդական թվով բաժանումն է. օրինակ՝ $5/2 = 2.5$:
2. `//` -սա ամբողջ մասով բաժանումն է. այս դեպքում բաժանման արդյունքը լինում է միայն ամբողջ մասը, օրինակ՝ $8//3 = 2$:
3. `%` -սա մնացորդով բաժանումն է. այս դեպքում վերադարձվում է բաժանման արդյունքում առաջացած մնացորդը, օրինակ՝ $5\%2 = 1$:

Երեք գործողության դեպքում էլ հասանելի է կիրառման վերոնշյալ ձևերը: Կարևոր է հիշել, որ ինչպես մաթեմաթիկայում թիվը չի կարելի բաժանել 0-ի, այնպես էլ այստեղ դա կհանգեցնի ծրագրի սխալի, և այն կդադարի:

Նշում. ծրագրավորման մեջ `#` սիմվոլից հետո գրվածը չի հանդիսանում ծրագրի մաս: Դա մեկնաբանություն է, որը նախատեսված է՝ մարդու կողմից ընթերցելու համար: Մեկնաբանություններն օգտագործվում են՝ կոդը պարզաբանելու, բացատրություններ տալու կամ հիշեցումներ թողնելու համար:

Տասնորդական թվով բաժանում

Ամբողջ մասով բաժանում

Մնացորդով բաժանում

Այս բոլոր գործողությունները նույնությամբ գործում են նաև տասնորդական (float) թվերի համար, տրամաբանական (bool) տիպի փոփոխականների համար, որտեղ True արժեքը

հավասար է 1-ի, իսկ False արժեքը՝ 0-ի: Օրինակ՝

Տեքստային փոփոխականների դեպքում մաթեմատիկական գործողություններից հասանելի են միայն գումարումը և բազմապատկումը:

Գումարում գործողությունը հնարավորություն է տալիս միավորել 2 տեքստ:Ստորև բերված

օրինակում՝ ծրագիրը կտալի hello world:

Արտադրյալ գործողությունը հնարավորություն է տալիս կրկնել տեքստը մի քանի անգամ:
Ստորև բերված օրինակում՝ ծրագիրը կտալի barevbarevbarev

Այս դեպքում կրկին հասանելի են գումարում և բազմապատկում գործողությունների
կիրառման բոլոր ձևերը:

Փոփոխականի մեկ տիպից մյուսի անցում

Ճրագրավորման ժամանակ հաճախ անհրաժեշտ է լինում փոփոխականի մեկ տիպից անցնել
մյուսին:Python-ում դա կատարվում է հենց փոփոխականի անունը տիպի անվանման
օգնությամբ, սակայն կան որոշակի սահմանափակումներ: Փոփոխություն կատարելու համար
գրվում է ցանկալի վերջնական տիպը և փակագծերում նշվում է արժեքը, որի տիպը հարկավոր
է փոխել: Արժեքը կարող է լինել թե՛ փոփոխականի, թե՛ բուն արժեքի տեսքով: Ստորև բերված
է աղյուսակ, որտեղ ներկայացված են Python-ի հիմնական տվյալների տիպերից (int, float, str,
bool) միմյանց փոխարկման հնարավորությունները, սահմանափակումները և
առանձնահատկությունները:

Սկզբնական տիպ	Նպատակային տիպ	Հնարավոր է փոխարկումը	Առանձնահատկություններ / Սահմանափակումներ
int	float	Այո	Անմիջապես փոխվում է, օրինակ՝ int(5) → 5.0:
int	str	Այո	Արժեքը դառնում է տեքստ՝ str(5) →

			'5':
int	bool	Այո	0 → False, ցանկացած այլ թիվ → True:
float	int	Այո	Կտրված է տասնորդական մասը՝ int(5.9) → 5:
float	str	Այո	Դառնում է տեքստ՝ str(3.14) → '3.14':
float	bool	Այո	0.0 → False, այլ արժեք → True
str	int	Միայն թվային տողերի համար	Պետք է լինի ամբողջ թիվ՝ int('42') → 42:
str	float	Միայն թվային տողերի համար	Պետք է լինի տասնորդական թվի տեսքով՝ float('3.14'):
str	bool	Այո	Դատարկ տող → False, այլ տող → True:
bool	int	Այո	True → 1, False → 0:
bool	float	Այո	True → 1.0, False → 0.0:
bool	str	Այո	str(True) → 'True'

Կարևոր է նաև հիշել նշանով նշված դեպքերում ճիշտ մուտքային արժեք չտալու դեպքում ծրագիրը կհանգի ValueError-ի և կդադարեցվի: Ստորև կարող եք տեսնել միջանի օրինակներ՝

int -> float

float -> int

int -> str

str -> int

str -> float

bool -> int

int -> bool

Թվային փոփոխականը տրամաբանական դարձնելիս ցանակացած արժեք, բացի 0-ից, կլինի True իսկ 0-ն False:

Իսկ տեքստի դեպքում դատարկ տեքստը False իսկ եթե ունի պարունակություն ապա True:

str -> bool

Փոփոխականի արժեքի ներմուծումը օգտատիրոջ կողմից

Մինչ այս փոփոխականի արժեքները դրված են եղել մեր կողմից հենց ծրագրի մեջ, սակայն Python-ում առկա է հնարավորություն տվյալները ներմուծել դա կատարվում է input() հրամանի օգնությամբ: input() հրամանի փակագծերում կարող է լինել տեքստ, որը ցույց կտա, թե ինչպիսի տվյալներ պետք է մուտքագրվեն: input()-ը մեզ տրամադրում է միայն տեքստային (str) տիպի տվյալներ, իսկ եթե մեզ հարկավոր լինի ներմուծել այլ տիպի տվյալներ, հարկավոր է այն ներմուծել որպես str, այնուհետև ձևափոխել մեզ անհրաժեշտ տիպի (int, bool, float):

```
===== RESTART: C:/Users/GenAI
Entr Text and it will be printed
```

Տեսքը ծրագիրը աշխատացնելիս այն կլինի այնպիսին, որ անհրաժեշտ կլինի մուտքային տվյալներ ներմուծել (թե՛ input հրաման գրված լինի, թե՛ որքան`

այդքան տվյալներ պետք է ներմուծվեն):

```
===== RESTART: C:/Users/GenAI/Test.py
Entr Text and it will be printed This is my Entered Text
This is my Entered Text
```

Ծրագիրը ստանալով մուտքային տվյալը այն պահապանում է փոփոխականում այնուհետև տպում այն: input() հրամանը կարելի է կիրառել նաև առանձ տեքստի: Այս դեպքում ծրագիրը գործարկելուց հետո անմիջապես հարկավոր է ներմուծել մուտքային տվյալները:

Այս դեպքում հարկավոր է ներմուծել ամբողջ թիվ, և ծրագիրը կտաի ներմուծված թվին

գումարած 6 արտահայտության արժեքը:

```
===== RESTART: C:/Users,
4
10
```

Ահա ծրագրի արդյունքը:

Սխալների առաջացում

Հարկավոր է նաև հիշել, որ փոփոխականի մեկ տիպից մյուսի անցնելիս հարկավոր է լինել ուշադիր, քանի որ կարող է առաջանալ error (սխալներ), երբ մենք ցանկանանք փոխել մեկ տիպից մյուսի, և դա հնարավոր չլինի: Օրինակ՝ երբ փորձենք տեքստ ձևափոխել թվի: Ստորև բերված է օրինակ, որտեղ առաջանում է ValueError: Հիշեք, որ այդ դեպքում ծրագիրը

դադարում է, այսինքն՝ չեն կատարվելու նաև այդ տողից հետո գտնվող գործողությունները:

Ծրագրի աշխատանքի արդյունք՝

```
Traceback (most recent call last):  
  File "C:/Users/Gor Hovsepyan/Documents/1.py", line 2, in <module>  
    num = int(text)  
ValueError: invalid literal for int() with base 10: 'Hello'  
|
```

Այս օրինակում կարող ենք տեսնել, քանի որ ծրագիրը չի կարող "Hello" բառը ձևափոխել թվի, այն դադարում է և ցույց տալիս համապատասխան սխալ (արժեքի սխալ՝ ValueError):

Առաջադրանքներ

1. Ստեղծիր ամբողջ թիվ պարունակող x փոփոխական և տպիր դրա արժեքը:
2. Ստեղծիր տեքստ պարունակող myText փոփոխական և տպիր դրա արժեքը:
3. Ստեղծիր տրամաբանական (bool) տիպի x փոփոխական, որի մեջ պահված լինի True, և տպիր դրա արժեքը:
4. Ստեղծիր փոփոխական, որի մեջ պահված կլինի մի թիվ, գումարի մեկ այլ թիվ և տպիր արդյունքը:
5. Ստեղծիր փոփոխական, որի մեջ պահված կլինի մի թիվ, գումարի մեկ այլ թիվ, պահիր նոր փոփոխականում և տպիր արդյունքը:
6. Ստեղծիր տասնորդական թիվ պարունակող փոփոխական, բազմապատկիր այն մեկ այլ թվով, արդյունքը պահիր նույն փոփոխականի մեջ և տպիր արդյունքը:
7. Ստեղծիր թիվ պարունակող փոփոխական և տպիր դրա ամբողջ մասը:
8. Ստեղծիր թիվ պարունակող փոփոխական և տպիր դրա ամբողջ մասը և բաժանման մնացորդը (օրինակ՝ 5-ը բաժանած 2-ի):
9. Ստեղծիր թիվ պարունակող փոփոխական և տպիր դրա խորանարդը (power of 3):
10. Ստեղծիր երկու տեքստային փոփոխական, միավորիր դրանք, պահիր առաջին փոփոխականի մեջ և տպիր այն:

11. Ստեղծիր թիվ պարունակող և տեքստ պարունակող փոփոխականներ, միավորիր դրանք նոր տեքստային փոփոխականում և տպիր այն:
12. Ստեղծիր տասնորդական թիվ պարունակող փոփոխական և տպիր դրա միայն ամբողջ մասը:
13. Գրի ծրագիր, որը կհաշվի և կտպի հետևյալ արտահայտության արժեքը՝ $5 * x + x ** 2 - True$, որտեղ x -ը ամբողջ թվային փոփոխական է:
14. Գրիր ծրագիր, որը հարցնում է օգտագործողի անունը և տպում ողջունի հաղորդագրություն: Օրինակ՝ Hello, Gor!
15. Գրիր ծրագիր, որը մուտքում ստանում է երկու թիվ, գումարում է դրանք և տպում արդյունքը:
16. Գրիր ծրագիր, որը մուտքում ստանում է մարդու տարիք և տպում է թե որ թվականին է նա ծնվել:

Թեմա 2. Տողեր (String)

Տեքստը մարդկանց հաղորդակցման հիմնական միջոցներից մեկն է: Ծրագրավորման մեջ էլ նույն կերպ՝ տողերը հանդիսանում են տվյալների ամենահին և ամենակարևոր տեսակներից մեկը:

Երբ առաջին համակարգիչները ստեղծվեցին՝ նրանք աշխատում էին միայն թվերով: Սակայն շուտով անհրաժեշտ դարձավ մուտքագրել, պահել և մշակել բառեր, նախադասություններ, օգտատերերի անուններ, գաղտնաբառեր և այլն: Այսպիսով, հայտնվեցին տողային տվյալների տիպերը:

- 1960-ականներին ALGOL և COBOL լեզուներն արդեն ունեին primitive տիպեր տեքստերի համար:
- C լեզվում տողերը ներկայացվում էին սիմվոլների զանգվածով (char array), որտեղ յուրաքանչյուր տառ էր պահվում հիշողության մեջ՝ իր ASCII արժեքով:
- Python-ում՝ ի տարբերություն շատ այլ լեզուների, տողերը համարվում են օբյեկտներ՝ հզոր մեթոդներով և ֆունկցիաներով:

Ժամանակի ընթացքում string-ները դարձան բոլոր լեզուներում անհրաժեշտ կառուցվածք, քանի որ մարդկային հաղորդակցությունը առանց տեքստի գրեթե անհնար է:

Տողը (string) տվյալների այն տեսակն է, որը պարունակում է տեքստային տեղեկություն: Python-ում տողը ներկայացվում է Unicode սիմվոլների հաջորդականությամբ: Տող տիպի ինֆորմացիա պարունակող փոփոխականը Python-ում ստեղծվում է նույն կերպ, ինչպես և թիվ պարունակողը, սակայն որպեսզի ծրագիրը հասկանա, որ դա տող է, հարկավոր է այն ներառել ' ' կամ " " չափերտների մեջ, իսկ եթե փոփոխականը բաղկացած է միջանի տողերից կարող

ենք օգտագործել ''' ''' տարբերակը: Ստորև կարող եք տեսնել օրինակ:

Այս օրինակում մենք ստեղծեցինք տեքստային (Տող) տիպի փոփոխական: Համակարգչում բոլոր տվյալները պահպանվում են 0 և 1-երի օգնությամբ, տողերը բացառությունն չեն կազմում, և որպեսզի կարողանանք պահպանել այդ տվյալները, յուրաքանչյուր սիմվոլի համապատասխանեցվել է իր թիվը, որ աշխատում է ըստ ASCII աղյուսակի:

0	NUL	16	DLE	32		48	0	64	@	80	P	96	`	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	\$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	BEL	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	BS	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	HT	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	LF	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	VT	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	FF	28	FS	44	,	60	<	76	L	92	\	108	l	124	
13	CR	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	DEL

Ascii աղյուսակ

Նայելով այս աղյուսակին՝ կարող ենք տեսնել, որ, օրինակ, թվերը որպես սիմվոլներ և իրենց համապատասխանեցված թվերը իրար հետ չեն համընկնում: Այդ իսկ պատճառով էլ մենք չենք կարող թիվ պարունակող տեքստային փոփոխականին գումարել թիվ և արդյունքում ստանալ թիվ: Ստորև բերված օրինակում մենք կարող ենք տեսնել, որ ծրագրի աշխատանքի

արդյունքում կառաջանա սխալ (error), նշելով, որ հնարավոր չէ գումարել int և str տիպի փոփոխականներ:

Արդյունքը՝

```
Traceback (most recent call last):  
  File "C:/Users/Gor Hovsepyan/Documents/1.py", line 2, in <module>  
    print(Test+1)  
TypeError: can only concatenate str (not "int") to str
```

Տեքստային տիպի փոփոխականների համար գումարման գործողությունը աշխատում է որպես կոնկատենացիայի միջոց, այսինքն՝ տեքստերը միավորելու: Ստորև բերված օրինակում մենք գումարում ենք "Hello" և "python" բառերը՝ ստանալով նոր տեքստային փոփոխական, որը իր մեջ պահում է "Hello python" տեքստը:

Իսկ արտադրյալի գործողությունը հնարավորություն է տալիս կրկնել տեքստը: Ստորև բերված օրինակում կրկնում ենք "hello" բառը 10 անգամ:

Ճրագը աշխատանքի արդյունքը կարող էք տեսնել ստորև:

```
===== RESTART: C:/Users/GenAi / Test.py =====  
hello hello hello hello hello hello hello hello hello  
>>>
```

Մյուս մաթեմաթիկական գործողությունները տեքստային տիպի համար հասանելի չեն:

Տողի տարրերի հետ աշխատանք

Ինչպես ասվեց, տողը իրենից ներկայացնում է շարք, և դա հնարավորություն է տալիս ստանալ հասանելիություն տարրերից յուրաքանչյուրին՝ նույն կերպ, ինչպես և ցուցակներում: Հաշվարկը սկսվում է 0-ից: Ստորև կարող եք տեսնել օրինակը, որտեղ տպում ենք տեքստի 2-րդ տառը:

Արդյունքը՝

```
===== RESTART: C:/Users/GenAi / Test.py =====  
e  
>>>
```

Ինչպես նաև կարող ենք տպել վերջից: Այս դեպքում հարկավոր է գրել թիվը վերջից հաշված՝ միևնույն: Օրինակ՝ վերջինը դա -1-ն է:

Արդյունքը՝

```
===== RESTART: C:/Users/GenAi / Test.py =====  
>>> |
```

Python-ը հնարավորություն է տալիս ստանալ նաև որոշակի միջակայքը օրինակ 2-5 միջակայքում ընկած տարրերը իդեա կարելի է հիշել որ միջակայքը կիսաբաց է այսինքն վերջին նշված համարի ներքո գտնվող տարրը չի ներառվում: Այս դեպքում օգտագործվում է հետևյալ գրելաձևը տեքստային ֆոփոխական[Սկիզբ : Վերջ]

Արդյունքը՝

```
===== RESTART:  
men  
>>> |
```

Հարկավոր է հիշել, եթե միջակայքի վերջը լինի ավելի մեծ, քան երկարությունը, ապա ծրագիրը կտալի բառը մինչև վերջ: Նույն արդյունքը կունենանք, եթե վերջակետի հետո թիվ չգրենք:

Արդյունքը՝

```
===== RESTART:  
menia  
>>> |
```

Նույն կերպ եթե չնշենք սկիզբը ապա կտալի սկզբից մինչև մեր նշված հատվածը:

```
===== RESTART:  
Armen  
>>> |
```

Ինչպես նաև ինդեքսների հետ աշխատելիս հարկավոր է նշել, որ եթե նշենք տարրերը հակառակ հերթականությամբ, ապա ծրագիրը ոչինչ չի տալի: Իսկ եթե ցանկանում ենք շրջել տեքստը, ապա կարող ենք կիրառել [::-1] տարբերակը:

Արդյունքը՝

```
===== RESTART:
ainemrA
>>>
```

Կրկնակ վերջակետը իրականում օգտագործվում է բաժանման համար ըստ անհրաժեշտ քայլի, այն շատ մնամ է range հրամանին այն ունենում է հետևյալ կառուցվածքը [Սկիզ : Վերջ: քայլ] ծրագիրը վերցնելու է մեր միջակայքում գտնվող տարրերը ըստ նվաճ քայլի

Արդյունք՝

```
===== RESTART:
Amn
>>>
```

Տեքստային տիպի համար հասանելի ներդրված ֆունկցիաներ

Ինֆորմատիվ (տեղեկություն վերադարձնող) մեթոդներ

Հրաման / մեթոդ	Բացատրություն	Օրինակ
len(text)	Վերադարձնում է տողի երկարությունը (սիմվոլների քանակը)	len("Barev") → 5
text.count("a")	Հաշվում է նշված սիմվոլի կամ տողի քանակը տողի մեջ	"banana".count("a") → 3
"a" in text	Ստուգում է՝ արդյոք նշված սիմվոլը կա տողի մեջ	"a" in "barev" → True
text.find("a")	Վերադարձնում է առաջին հանդիպման ինդեքսը (եթե չկա՝ -1)	"barev".find("e") → 3
text.index("a")	Նույնը՝ ինչ find(), բայց սխալ է տալիս, եթե չի գտնվել	"barev".index("e") → 3
text.startswith("b")	Ստուգում է՝ սկսվո՞ւմ է նշված տառով	"barev".startswith("b") →

		True
<code>text.endswith("v")</code>	Ստուգում է՝ ավարտվո՞ւմ է նշված տառով	<code>"barev".endswith("v") → True</code>

Ստուգման մեթոդներ (Կերպարվում են boolean)

Հրաման / մեթոդ	Բացատրություն	Օրինակ
<code>text.isdigit()</code>	Ստուգում է՝ արդյոք բոլոր սիմվոլները թվեր են	<code>"123".isdigit() → True</code>
<code>text.isalpha()</code>	Ստուգում է՝ արդյոք բոլոր սիմվոլները տառեր են	<code>"barev".isalpha() → True</code>
<code>text.isalnum()</code>	Տառ կամ թիվ՝ առանց բացատների	<code>"abc123".isalnum() → True</code>
<code>text.isspace()</code>	Ստուգում է՝ արդյոք ամբողջ տողը միայն բացատ է	<code>" ".isspace() → True</code>
<code>text.isupper()</code>	Ստուգում է՝ բոլոր տառերը մեծատառ՞ն են	<code>"HELLO".isupper() → True</code>
<code>text.islower()</code>	Ստուգում է՝ բոլոր տառերը փոքրատառ՞ն են	<code>"hello".islower() → True</code>
<code>text.istitle()</code>	Ստուգում է՝ յուրաքանչյուր բառ սկսվո՞ւմ է մեծատառով	<code>"Hello World".istitle() → True</code>

Փոփոխման մեթոդներ

Հրաման / մեթոդ	Բացատրություն	Օրինակ
<code>text.upper()</code>	Բոլոր տառերը դարձնում է մեծատառ	<code>"python".upper() → "PYTHON"</code>
<code>text.lower()</code>	Բոլոր տառերը դարձնում է փոքրատառ	<code>"HELLO".lower() → "hello"</code>
<code>text.capitalize()</code>	Միայն առաջին տառը մեծատառ է դարձնում	<code>"hello".capitalize() → "Hello"</code>
<code>text.title()</code>	Ամեն բառի առաջին տառը դարձնում է մեծատառ	<code>"barev ashxarh".title() → "Barev Ashxarh"</code>
<code>text.swapcase()</code>	Փոխում է մեծատառը փոքրատառով և հակառակը	<code>"HeLlO".swapcase() → "hElIo"</code>
<code>text.replace("a", "o")</code>	Փոխարինում է բոլոր "a"-ները "o"-ով	<code>"banana".replace("a","o") → "bonono"</code>
<code>text.strip()</code>	Հեռացնում է բացատները և \n/\t տողի սկզբից և վերջից	<code>" hi ".strip() → "hi"</code>
<code>text.lstrip()</code>	Հեռացնում է միայն ձախ կողմի բացատները	<code>" hi".lstrip() → "hi"</code>
<code>text.rstrip()</code>	Հեռացնում է միայն աջ կողմի	<code>"hi ".rstrip() → "hi"</code>

	բացատրները	
--	------------	--

Բաժանում և միացում

Հրաման / մեթոդ	Բացատրություն	Օրինակ
<code>text.split()</code>	Տողը բաժանում է բառերի ցուցակի ըստ բացատի	<code>"hi there".split() → ["hi", "there"]</code>
<code>text.split(",")</code>	Բաժանում է ըստ նշված սիմվոլի	<code>"a,b,c".split(",") → ["a", "b", "c"]</code>
<code>text.splitlines()</code>	Բաժանում է ըստ տողի վերջի (newline)	<code>"hi\nbye".splitlines() → ["hi", "bye"]</code>
<code>" ".join(list)</code>	Ցուցակի տարրերը միավորում է մեկ տողի մեջ	<code>" ".join(["hi","there"]) → "hi there"</code>

Նաև հարկավոր է հիշել որ տողերի հետ հնարավոր է աշխատել ցիկլերով նույն կերպ ինչպես և ցուցակների հետ :Ստորև բերված օրինակում հաշվում ենք բառում a տառի քանակը:

Առաջադրանքներ

- 1. Մուտքագրիր տեքստ և տպիր՝ քանի սիմվոլ կա նրա մեջ:
- 2. Մուտքագրված տեքստը վերածիր ամբողջությամբ փոքրատառերի:
- 3. Մուտքագրված տեքստը վերածիր ամբողջությամբ մեծատառերի:
- 4. Մուտքագրված տեքստը փոխիր այնպես, որ միայն առաջին բառի առաջին սիմվոլը լինի մեծատառ, իսկ մնացածը՝ փոքրատառ:
- 5. Ստուգիր՝ արդյոք մուտքագրված տեքստը սկսվում է «A» սիմվոլով:
- 6. Հաշվիր, թե քանի անգամ է տեքստում հանդիպում «k» սիմվոլը:
- 7. Ստուգիր՝ արդյոք տեքստի մեջ առկա է «Python» բառը:
- 8. Ստուգիր՝ արդյոք մուտքագրված տեքստը ավարտվում է բացականչական նշանով:
- 9. Մուտքագրիր նախադասություն և բաժանիր այն առանձին բառերի:

10. Տրված մի քանի բառերից ձևավորիր մեկ ամբողջական նախադասություն՝ տեղադրելով բացատներ:
11. Գտիր, թե որ դիրքում է տեքստում առաջին անգամ հանդիպում «e» սիմվոլը:
12. Փոխիր տեքստը այնպես, որ բոլոր «a» սիմվոլները փոխարինվեն «@»-ով:
13. Մուտքագրված տեքստից հեռացրու բացատները սկզբից և վերջից:
14. Մուտքագրիր երկու տեքստ և համեմատիր՝ արդյոք դրանք միանման են:
15. Ստուգիր՝ արդյոք մուտքագրված տեքստը կազմված է միայն թվանշաններից:
16. Մուտքագրված նախադասության յուրաքանչյուր բառի առաջին սիմվոլը դարձրու մեծատառ:
17. Տպիր մուտքագրված տեքստը հակառակ ուղղությամբ:
18. Հաշվիր, թե քանի ձայնավոր սիմվոլ կա մուտքագրված տեքստում:
19. Ստուգիր՝ արդյոք մուտքագրված բառը պալինդրոմ է (կարդացվում է նույնը աջից և ձախից):
20. Մուտքագրված տեքստից հեռացրու նշված հատվածը, եթե այն գտնվում է սկզբում:

Թվային օպերատորներ

Մաթեմատիկական մարդկանց մտածողության մեջ գոյություն ունի հազարավոր տարիներ. թվերի գումարումը, հանումը, բաժանումը և բազմապատկումը կիրառվել են առօրյա կյանքում դեռևս առաջին քաղաքակրթություններից:

Օրինակ՝

- Առևտրի ժամանակ ապրանքների գինը հաշվելիս կիրառվում է գումարում և բազմապատկում:
- Սնունդը բաժանելիս՝ բաժանում:
- Տարբեր չափեր համեմատելիս՝ հանում:
-

Python-ում այս նույն գործողությունները իրականացվում են թվային օպերատորներով:

Հիմնական թվային օպերատորները

Օպերատոր	Նշանակություն	Օրինակ	Արդյունք
+	Գումարում	5 + 3	8
-	Հանում	10 - 4	6
*	Բազմապատկում	7 * 6	42
/	Բաժանում (float)	9 / 2	4.5
//	Բաժանում ամբողջ թվով	9 // 2	4

%	Մնացորդ	9 % 2	1
**	Աստիճան	2 ** 3	8

Օրինակներ

Թվային օպերատորների առաջնահերթություններ

Երբ միևնույն արտահայտության մեջ օգտագործվում են տարբեր օպերատորներ, Python-ը նախ հաշվում է ավելի բարձր առաջնահերթություն ունեցող գործողությունները: Եթե օպերատորները ունեն նույն առաջնահերթությունը, ապա հաշվարկը կատարվում է ձախից աջ:

Առաջնահերթություն	Օպերատորներ	Նշանակություն	Օրինակ	Արդյունք
1	()	Կլոր փակագծեր	(2 + 3) * 4	20
2	**	Աստիճան	2 ** 3 ** 2	512
3	-x	Միաթև նշաններ	-5 + 3	-2
4	*, /, //, %	Բազմապատկում և բաժանում	10 / 2 * 3	15.0
5	+, -	Գումարում և հանում	5 - 2 + 1	4

Հիմնական կանոններ:

1. Փակագծերը միշտ ունեն ամենաբարձր առաջնահերթություն:
2. Աստիճանը հաշվարկվում է աջից ձախ՝
Օրինակ՝ $2 ** 3 ** 2 = 2 ** (3 ** 2)$:
3. Բազմապատկում, բաժանում, ամբողջ բաժանում և մնացորդ՝ նույն մակարդակի են, հաշվում են ձախից աջ:
4. Գումարում և հանում կատարվում են ամենավերջում, ձախից աջ:

Բացի օպերատորներից, Python-ը տրամադրում է նաև ներկառուցված ֆունկցիաներ՝ `abs()`, `pow()`, `divmod()`, `round()`:

- `abs(x)` → վերադարձնում է թվի բացարձակ արժեքը:
- `pow(x, y)` → վերադարձնում է x^y (նույնն է, ինչ $x ** y$):
- `divmod(a, b)` → վերադարձնում է զույգ (բաժանում, մնացորդ)
- `round(3.14159, n)` → կլորացնում է թիվը, ստորակետից հետո n նիշի ճշտությամբ

Նշված արտահայտությունները գրել փայթն լեզվի օպերատորների միջոցով:

1. Հաշվել, a , b թվերի գումարի խորհանարդը:
2. Հաշվել a թվի քառակուսու և b թվի արտադրյալը:
3. Հաշվել, a թվի b աստիճանի 7-ի բաժանելիս ստացվող մնացորդը:
4. Հաշվել a , b թվերի քանորդի և մնացորդի գումարը:
5. Հաշվել a -թվի բացարձակ արժեքի և b -ի տարբերությունը:
6. Հաշվել a -ի $b*c$ աստիճանը:

Նշանակման օպերատորներ

Փայթն լեզվում կան հատուկ օպերատորներ, որոնք նախատեսված են փոփոխականի արժեքի փոխման համար:

Օպերատոր	Նշանակություն	Օրինակ	Համարժեք
+=	Գումարում և վերագրում	$a += b$	$a = a + b$
-=	Հանում և վերագրում	$a -= b$	$a = a - b$
*=	Բազմապատկում և վերագրում	$a *= b$	$a = a * b$
/=	Բաժանում և վերագրում	$a /= b$	$a = a / b$
//=	Ամբողջ բաժանում և վերագրում	$a //= b$	$a = a // b$
%=	Մնացորդով բաժանում և վերագրում	$a \% = b$	$a = a \% b$
**=	Աստիճանի բարձրացում և վերագրում	$a ** = b$	$a = a ** b$

Նշված արտահայտությունները գրել փայթն լեզվի նշանակման օպերատորների միջոցով:

1. A թիվը մեծացնել 7-ով:
2. A թիվը փոքրցնել 3 անգամ:
3. A թիվը դարձնել $\square^2$:
4. A- թվին վերագրել իր վերջին թվանշանը:

Բիթային օպերատորներ*

Բիթային օպերատորները աշխատում են թվերի երկուական ներկայացման հետ:

Օպերատոր	Նշանակություն	Օրինակ	Արդյունք (բինար)
&	AND	5 & 3	1 (101 & 011 = 001)
	OR	5 3	7 (101 011 = 111)
^	XOR	5 ^ 3	6 (101 ^ 011 = 110)
~	NOT	~5	-6 (~00000101 = 11111010)
<<	Ձախ տեղաշարժ	5 << 1	10 (101 << 1 = 1010)
>>	Աջ տեղաշարժ	5 >> 1	2 (101 >> 1 = 10)

Բիթային օպերատորների մասին ավելի մանրամասն կարող եք կարդալ [այստեղ](#):

Առաջադրանքներ

- 1. Գրել ծրագիր, որը ստանում է երկու ամբողջ թիվ և տպում է դրանց գումարը:
- 2. Գրել ծրագիր, որը ստանում է երկու ամբողջ թիվ և տպում է դրանց տարբերությունը բացարձակ արժեքը:
- 3. Գրել ծրագիր, որը ստանում է երկու ամբողջ թիվ և տպում է դրանց արտադրյալի հակադիր թիվը:
- 4. Գրել ծրագիր, որը ստանում է երկու ամբողջ թիվ և տպում է առաջինի ամբողջ թվային բաժանման արդյունքը երկրորդի վրա:
- 5. Գրել ծրագիր, որը ստանում է երկու ամբողջ թիվ և տպում է առաջինի բաժանման մնացորդը երկրորդի վրա:
- 6. Գրել ծրագիր, որը ստանում է թիվ և տպում է այդ թվի քառակուսին:
- 7. Գրել ծրագիր, որը ստանում է երկու թիվ և տպում է դրանց միջինը:
- 8. Գրել ծրագիր, որը ստանում է չորս թիվ և տպում է դրանց գումարը և միջինը:
- 9. Գրել ծրագիր, որը ստանում է թիվ և տպում է վերջին թվանշանը:
- 10. Գրել ծրագիր, որը ստանում է թիվ և աջից կցագրում է 8 թիվը:

11. Գրել ծրագիր, որը ստանում է թիվ և ջնջում է վերջին թվանշանը:

12. Գրել ծրագիր, որը ստանում է թիվ և վերջին թվանշանը դարձնում է 5:

Թեմա 4. Ցիկլեր (Loops) For While

Առօրյայում հաճախ լինում են գործողություններ, որոնք կրկնվում են, քանի դեռ որոշակի պայմաններ պահպանվում են: Օրինակ՝ մենք ամեն առավոտ գնում ենք դպրոց, քանի դեռ արձակուրդ չէ, կամ մասնակցում ենք դասին, քանի դեռ զանգը չի հնչել:

Ծրագրավորման մեջ նման կրկնվող գործողությունները կատարելու համար գոյություն ունի *while* ցիկլը: Այն կատարում է որևէ գործողություն այնքան ժամանակ, քանի դեռ պայմանը ճիշտ է:

Այժմ դիտարկենք հենց դասին մասնակցելու օրինակը: Մենք ակտիվ մասնակցում ենք դասին այնքան ժամանակ, քանի դեռ այն չի ավարտվել (օրինակ՝ մինչև ժամը 14:30): Յուրաքանչյուր մասնակցությունից հետո ստուգում ենք ժամը, որպեսզի հասկանանք՝ արդյոք դասը ավարտվել է, թե դեռ շարունակվում է:

Ծրագրավորման տեսանկյունից սա կարելի է ներկայացնել *while* ցիկլով, որտեղ ժամը ստուգվում է ամեն ցիկլի ավարտին՝ որոշելու համար շարունակե՞լ մասնակցությունը, թե ոչ:

Վերոնշյալ ծրագրում մենք նշում ենք որ դասերը սկսվում է 8:30 և քանի դեռ ժամը 14:30 չէ մենք տպում ենք Keep learning և ևս մեկ ժամ մասնակցում ենք դասին և այդպես այնքնա ժամանակ քանի դեռ ժամը չի դառել 14:30 կամ ավելի մեծ իսկ երբ պայամը դառնում է սխալ այսինքն դասերը ավարտվում ենք ծրագիրը կտպի Classes are over: Ստորև կարող եք տեսնել ծրագրի աշխատանքի արդյունքը:

```
>>> |
===== RESTART: |
Keep learning
Keep learning
Keep learning
Keep learning
Keep learning
Keep learning
Keep learning
Classes are over
>>> |
```

Օրինակ՝ երբ անհրաժեշտ է ցուցակի բոլոր տարրերը մեծացնել 5-ով, ապա առանց ցիկլի ստիպված կլինենք ամեն տարրի համար առանձին գումարման գործողություն կատարել: Սակայն այս խնդիրը կարելի է շատ ավելի հեշտ լուծել while ցիկլի միջոցով:

Օրինակ առանց ցիկլի կիրառման՝

Օրինակ ցիկլի կիրառմամբ՝

Ինչպես տեսանք վերոնշյալ օրինակում, մենք կրճատեցինք ծրագիրը՝ դարձնելով այն ավելի ընթերցելի և հեշտ ընդլայնվող: Նախորդ դեպքում, երբ ցուցակ (list) մեծանում էր, անհրաժեշտ էր յուրաքանչյուր տարրի համար գրել առանձին տող՝ պահպանելու համար ծրագրի ճիշտ աշխատանքը: Իսկ while ցիկլի դեպքում ծրագիրը մնում է նույնը՝ անկախ զանգվածի տարրերի քանակից:

Ինչպես նշվեց, while ցիկլը կատարում է գործողությունները այնքան ժամանակ, քանի դեռ մեր կողմից գրված պայմանը ճիշտ է: Բայց ի՞նչ կլինի, եթե այդ պայմանը **միշտ լինի ճիշտ**, օրինակ՝ $0 == 0$ կամ պարզապես True: Այդ դեպքում ծրագիրը կմտնի **անվերջ ցիկլի**, որը կաշխատի այնքան ժամանակ, քանի դեռ ծրագիրը կամ համակարգիչը դադարեցված չէ: Ստորև կարող եք տեսնել անվերջ ցիկլի օրինակ.

Այս ծրագիրը անվերջ կտալի "Program is running" տեքստը իդեալ True- ի փոխարեն եթե գրենք թիվ ապա 0-ից տարբեր ցանկացած թիվ կդիտարկվի որպես ճիշտ և ցիկլը կկատարվի անվերջ իսկ 0 կդիտարկվի որպես False և ցիկլը չի կատարի իրեն վերագրված գործողությունները:

Սակայն իրական կյանքում հաճախ անհրաժեշտ է դառնում դադարեցնել ցիկլի աշխատանքը ոչ միայն հիմնական (while-ի) պայմանից կախված, այլ նաև **արտակարգ** որևէ իրավիճակի դեպքում: Օրինակ՝ դասերի ընթացքում կարող է հնչել տազնապի ազդանշան, որի դեպքում պետք է անհապաղ դադարեցնել գործընթացը:

Այդպիսի դեպքերում օգտագործվում է break հրամանը:

break հրամանի կիրառման դեպքում ծրագիրը դուրս է գալիս ցիկլից՝ անկախ այն բանից՝ արդյոք հիմնական պայմանը դեռ ճիշտ է, թե ոչ: Ցիկլից դուրս գալուց հետո կշարունակվեն հաջորդ գործողությունները:

Նշում. Եթե break հրամանը գրվի ցիկլից դուրս, ապա ծրագիրը կտա սխալ (SyntaxError), քանի որ այդ հրամանը նախատեսված է միայն ցիկլերի համար (while կամ for):

Ստորև բերված է նախորդ՝ դասերի մասնակցության օրինակը, սակայն այս անգամ ավելացվել է alarm անունով տրամաբանական (bool) փոփոխական: Ցիկլի յուրաքանչյուր կրկնման ժամանակ օգտատիրոջից մուտք է ստացվում alarm արժեքը: Եթե մուտքագրված

արժեքը լինի True, ապա ծրագիրը կդադարեցնի իր աշխատանքը՝ դուրս գալով ցիկլից: Այս օրինակում դասերը շարունակվում են, և յուրաքանչյուր դասից առաջ ներմուծում ենք «տազնապ» փոփոխականը: Եթե դրա արժեքը True է, ծրագիրը տպում է «Դասը դադարեցվել է տազնապի պատճառով» և դադարեցնում է ցիկլը: Այնուհետև կատարվում է ցիկլից դուրս գրված հրամանները՝ այս դեպքում տպվում է «Դասերը ավարտվել են կամ դադարեցվել են»: Եթե տազնապը մնում է False, ապա ծրագիրը շարունակվում է այնքան ժամանակ, քանի դեռ դասերի ժամը չի ավարտվել: Ավարտից հետո կրկին տպվում է «Դասերը ավարտվել են կամ դադարեցվել են»:

Ստորև կարող եք տեսնել ծրագրի աշխատանքի երկու տարբեր դեպքեր՝ առաջինում՝ երբ ծրագիրը չի ստանում տազնապ, երկրորդում՝ երբ ծրագիրը ստանում է տազնապ:

```

===== RESTART: C:/Users/
Enter alarm statusFalse
Keep learning
Enter alarm statusFalse
Keep learning
Enter alarm statusFalse
Keep learning
Enter alarm statusFalse
Keep learning
Enter alarm statusFalse
Keep learning
Enter alarm statusFalse
Keep learning
Classes are over
|

```

1)Երբ չի լինում տագնապ

```

===== RESTART: C:/Users:
Enter alarm statusFalse
Keep learning
Enter alarm statusFalse
Keep learning
Enter alarm statusTrue
Class stopped due to alarm!
Classes are over
|

```

2)Երբ տագնապ է լինում

Սա մեզ հնարավորություն կտա նաև դադարեցնել անվերջ ցիկլերը: Դիտարկենք եվս մեկ օրինակ երբ alarm փոփոխականը կստանանք ոչ թե input- ի օգնությամբ այլ կլինի նախապես

փիքսված արժեքով փոփոխական:

Ինչպես տեսանք վերոնշյալ օրինակներում, while ցիկլը հնարավորություն տվեց հեշտությամբ աշխատել կրկնվող գործողությունների հետ: Սակայն շատ հաճախ անհրաժեշտ է աշխատել iterable(օբյեկտ է, որն ապահովում է հաջորդական տարրերի վրա հերթով անցնելու հնարավորություն այդպիսի օբյեկտներից են list , set, tuple) հետ, և այդ դեպքում ավելի հարմար է օգտագործել for ցիկլը: for ցիկլը թույլ է տալիս հերթականությամբ ստանալ ցուցակի բոլոր տարրերը՝ առանց լրացուցիչ փոփոխական սահմանելու և յուրաքանչյուր քայլի ժամանակ այն փոփոխելու:

Ստորև ներկայացված է օրինակ, որտեղ ծրագիրը տպում է ցուցակի բոլոր տարրերը՝

յուրաքանչյուրը նոր տողում.

Այս դեպքում ծրագիրը տպում է անսուները յուրաքանչյուրը նոր տողում: Նույն գործողությունը կարելի է կատարել նաև օգտագործելով while ցիկլը սակայն ծրագիրը կլինի ավելի ծավալուն ստորև կարող եք տեսնել այդ օրինակը:

For ցիկլի վերոնշյալ օրինակում մենք կարող ենք միայն վերցնել արժեքները և չենք կարող փոփոխել ցուցակի արժեքները դա կատարելու համար կորդ ենք օգտագործել for ցիկլը և range հրամանը վերցնելով ոչ թե ցուցակի տարի արժեքը այլ նրա հմարանը:

Այս օրինակում մենք փոփոխում ենք ցուցակի տարրերը նրանցից յուրաքանչյուրում պահելով իր սկզբնական արժեքի քառակուսին:

Ինչպես և while ցիկլի դեպքում for ցիկլի աշխատանքը կրկին կարելի է դադարեցնել օգտագործելով break հրամանը:

Հաճախ անհրաժեշտ է լինում ցիկլի ավարտից հետո կատարել որոշակի գործողություններ և դա կատարելու համար python լեզվում առկա է for else և while else կառուցվածքները: Այս կառուցվածքների դեպքում else հրամանին վերագրված գործողությունները կատարվում են միայն այն ժամանակ երբ ցիկլը ավարտվել է առանձնապես break հրամանի աշխատանքի: Ստորև բերված է օրինակներ

Վերոնշյալ օրինակում ցիկլը կտալի 0,1,2 որից հետո ցիկլի աշխատանքը կդադարեցվի ևսկ ստորը բերված օրինակում աշխատանքի վերջում ծրագիրը կտալի 0-ից մինչև 4 թիվը

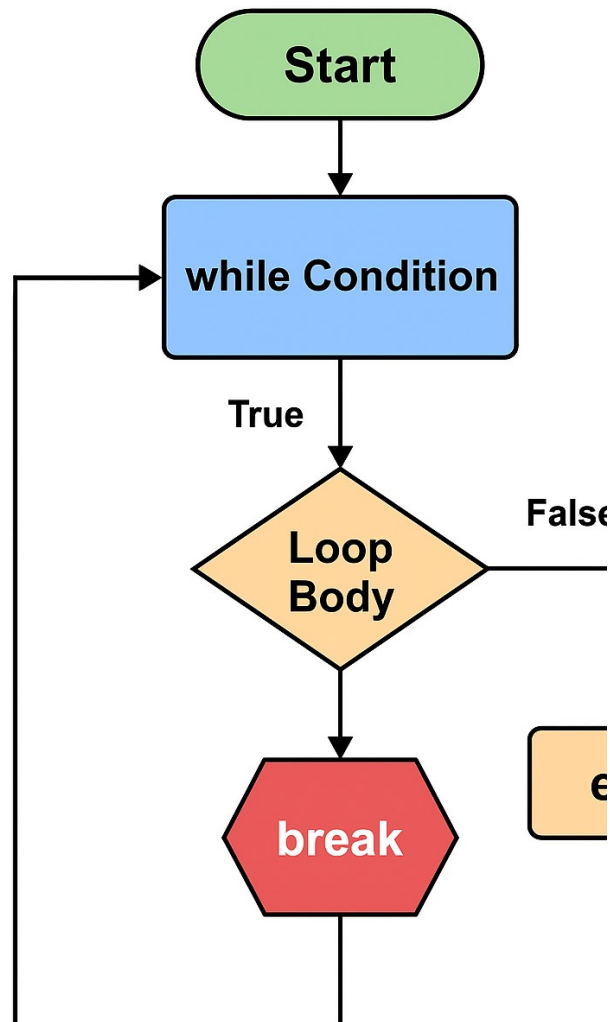
այնուհետև "For loop completed without break." :

Այժմ եկեք դիտարկենք while else – ի կիրառման օրինակ: Այս օրինակում մենք ստանում ենք

մուտքային թիվ և պետք է ասենք այն պարզ է թե ոչ:

Այս օրինակում մենք ներմուծում ենք թիվ և ցիկլի օգնությամբ ստուգում ենք արդյոք թիվը բաժանվում է 2-ից մինչև մեր թիվը ընկած միջակայքի որևիցե թվի թե ոչ եթե այն բաժանվում է ապա ծրագիրը կտալի որ այն պարզ չէ և կդադարեցնի ցիկլը break հրամանով հետևաբար չի աշխատի else-ին վերագված ծրագիրը իսկ եթե թիվը չբաժանվի ոչ մի թվի այսինքն ցիկլը չի դադարեցվի որևէ այլ հրամանի կողմից

ապա կաշխատի else-ին վերագրված հատվածը և ծրագիրը կտալի որ թիվը պարզ է: Ստորև բերված նկարում կարող եք տեսնել ցիկլի աշխատանքի գծապատկերը:



Եկեք դիտարկենք ևս մեկ օրինակ while ցիկլի կիրառմամբ, որի միջոցով որոշում ենք՝ արդյոք տրված թիվը պարզ է, թե ոչ:

Ստորև բերված օրինակում մենք ներմուծում ենք մի թիվ և ստուգում. եթե այն փոքր կամ հավասար է 1-ի, ապա տպում ենք, որ թիվը պարզ չէ: Հակառակ դեպքում՝ ստեղծում ենք բաժանարար փոփոխական և բուլյան (Boolean) տիպի is_prime փոփոխական, որի սկզբնական արժեքը դնում ենք True:

Այնուհետև, քանի դեռ բաժանարարը փոքր է մեր թվից, ստուգում ենք՝ արդյոք թիվը բաժանվում է բաժանարարին առանց մնացորդի: Եթե այո, նշանակում է թիվը պարզ չէ, այդ դեպքում is_prime-ը դարձնում ենք False և դադարեցնում ցիկլը:

Վերջում կրկին ստուգում ենք is_prime փոփոխականի արժեքը.

- Եթե այն մնացել է True, այսինքն՝ որևէ բաժանարար չի գտնվել, ապա թիվը պարզ է:
- Եթե դարձել է False, նշանակում է բաժանում տեղի է ունեցել, հետևաբար թիվը բաղադրյալ է:

Արդյունք՝

<pre>>>> Enter number5 prime number</pre>	<pre>>>> Enter number6 not prime</pre>
--	---

Ինչպես if-ի դեպքում, այնպես էլ ցիկլերի դեպքում կարող է լինել ներդրված ցիկլ, այսինքն՝ մեկ ցիկլին վերագրված գործողությունների մեջ լինի երկրորդ ցիկլ։ Ստորև բերված է 3x3 մատրիցի տպման ծրագիրը։

Առաջադրանքներ

1. Տպել 0-ից մինչև 9 թվերը՝ յուրաքանչյուրն առանձին տողում:
2. Գրել ծրագիր, որը մուտքում ստանում է թիվ և տպում է այդքան անգամ "Hello":
3. Հաշվել և տպել թվերի գումարը 1-ից մինչև 100:
4. Գտնել և տպել բոլոր գույգ թվերը 1-ից մինչև 50 միջակայքում:
5. Մուտքում ստանալ 10 ամբողջ թիվ և տպել դրանց միջին թվաբանականը:
6. Տպել բոլոր թվերը 100-ից 50 ներքև՝ հակառակ հերթականությամբ:
7. Տպել 1-ից մինչև 10 թվերի քառակուսիները:
8. Մուտքում ստանալ թիվ և պարզել՝ արդյոք այն պարզ թիվ է, թե ոչ:
9. Հաշվել թվանշանների քանակը մուտքագրված ամբողջ թվի մեջ:
10. Տպել թվի բոլոր բաժանարարները (առանց մնացորդի բաժանվող թվերը):
11. Հաշվել և տպել m -ից մինչև n թվերի արտադրյալը (factorial):
12. Գրել ծրագիր, որը մուտքում ստանում է տեքստ և տպում է այն n անգամ՝ յուրաքանչյուր անգամ նոր տողով:
13. Գտնել 1-ից մինչև 100 թվերի միջից այն թվերը, որոնք միաժամանակ բաժանվում են 3-ի և 5-ի վրա:
14. Հաշվել ու տպել մուտքագրված թվի թվանշանների գումարը:
15. Գտնել առաջին 20 կենտ թվերի գումարը և արտածել արդյունքը:
16. Գրել ծրագիր, որը տպում է 1-ից 10 բազմապատկման աղյուսակը հետևյալ ձևով՝

17. Մուտքում ստանալ 2 ամբողջ թիվ՝ տողերի և սյուների քանակը: Տպել աստղանիշերով

ուղղանկյուն համապատասխան չափսերով: Օրինակ

Թեմա 5. Պայմաններ (Conditions)

Առօրյա կյանքում մարդիկ իրենց կողմից կատարվող գործողությունները փոփոխում են՝ կախված որոշակի արտաքին պայմաններից: Օրինակ՝ դիտարկենք հագուստի ընտրությունը, որը կախված է եղանակային պայմաններից: Մարդը հագուստ ընտրելիս նայում է եղանակի կանխատեսումը և՛ կախված ջերմաստիճանից ու տեղումներից, որոշում է՝ ինչ հագուստ ընտրել և արդյոք վերցնել անձրևանոց, թե ոչ:

Նույն կերպ՝ ծրագրերի աշխատանքի ընթացքում ևս հարկավոր է փոփոխել ծրագրի վարքագիծը՝ կախված որոշ պայմաններից: Օրինակ՝ սոցիալական կայքերում գրանցվելիս, երբ մարդը նշում է իր տարիքային տվյալը, ծրագիրը ստուգում է՝ արդյոք նա 13 տարեկանից մեծ է, թե ոչ: Եվ՝ կախված այդ պայմանից, կարող են տեղի ունենալ երկու տարբեր որոշումներ՝

Եթե օգտատերը 13 տարեկան է կամ ավելի մեծ, ծրագիրը թույլ է տալիս գրանցվել:

Հակառակ դեպքում՝ ցուցադրվում է հաղորդագրություն այն մասին, որ տարիքային շեմը չի բավարարում գրանցման համար, և գրանցման գործընթացը դադարեցվում է:

Ծրագրավորման մեջ այդ գործընթացը կատարելու համար օգտագործվում է if պայմանի օպերատորը որը հնարավորություն է տալիս կատարել որևէ գործողություն եթե նշված

պայամը լինում է ճշմարիտ(True):Գրելաձև՝

if բանալի բառից հետո նշվում է պայման, որը կարող է լինել ճիշտ (True) կամ սխալ (False): Պայմանի ճիշտ լինելու դեպքում ծրագիրը կկատարի հաջորդ տողից սկսած՝ մեկ թաբ ներս գտնվող բոլոր հրահանգները, ապա կշարունակի աշխատանքը «պայմանից դուրս գործողություն» հատվածում: Եթե պայմանը սխալ է (False), ապա ծրագիրը անմիջապես կանցնի «պայմանից դուրս գործողություն» հատվածին:

Եթե պայմանում նշենք տրամաբանական տիպի փոփոխական (bool), ապա ծրագիրը կկատարի համապատասխան գործողությունը միայն այն դեպքում, երբ փոփոխականի արժեքը հավասար է True: Սակայն, բացի տրամաբանական տիպի փոփոխականից, կարող ենք օգտագործել նաև համեմատական օպերատորներ՝ պայմաններ կազմելու համար: Դրանք ներկայացված են ստորև՝ աղյուսակի տեսքով:

Օպերատոր	Նշանակությունը	Օրինակ	Արդյունք (True/False)
==	Հավասար է	5 == 5	True
!=	Հավասար չէ	4 != 3	True
>	Մեծ է	7 > 2	True
<	Փոքր է	3 < 1	False

Վերոնշյալ օրինակներում մենք քննարկեցինք այն դեպքերը, երբ պայմանը ճիշտ է՝ ծրագիրը կատարում է որևէ գործողություն: Սակայն իրական կյանքում հաճախ անհրաժեշտ է լինում՝ պայմանը ճիշտ լինելու դեպքում կատարել մեկ գործողություն, իսկ սխալ լինելու դեպքում՝ մեկ այլ: Հենց այդ դեպքերում մեզ օգնության է գալիս `else` հրամանը:

`else`-ը, ի տարբերություն `if`-ի, չի կարող գործել ինքնուրույն: Այն պետք է օգտագործվի `if` հրամանի հետ միասին: `else`-ը կատարում է իրեն վերագրված հրահանգները միայն այն դեպքում, երբ `if`-ում նշված պայմանը սխալ է (`False`):

Այս ծրագրում մուտքում ստանում ենք մարդու տարիքը և ստուգում ենք եթե այն մեծ է կամ հավասար 18-ի ապա տպում ենք `Capahas e` հակառակ դեպքում (երբ տարիքը փոքր է 18-ից) կաշխատի `else` -ին վերագրված հրամանները այսինքն կտպի `capahas che`:

Իրական կյանքում հաճախ անհրաժեշտ է լինում ստուգել մի քանի պայմաններ: Օրինակ՝ երբ դիտում ենք եղանակի կանխատեսությունը, ստուգում ենք՝ եթե դրսում անձրև է, վերցնում ենք անձրևանոցը, հակառակ դեպքում, եթե ձյուն է, հագնում ենք ձեռնոցներ, իսկ մյուս բոլոր դեպքերում վերցնում ենք արևապաշտպան գլխարկ: Այս դեպքում մեզ հարկավոր է ստուգել առաջին պայմանը, և միայն դրա չբավարարելու դեպքում ստուգել երկրորդը, իսկ եթե այն էլ չբավարարի, ապա միայն այդ դեպքում կիրականացվեն `else`-ին վերագրված գործողությունները:

«Հակառակ դեպքում եթե» հրամանը դա elif հրամանն է, որը, ի տարբերություն if-ի, չի կարող

գործել ինքնուրույն՝ առանց նախորդող if-ի: Այն կարող է հաջորդել նաև այլ elif հրամանների:

Վերոնշյալ օրինակում ծրագիրը նախ կստուգի՝ արդյոք եղանակը **Raining** է, թե ոչ: Քանի որ այս պայմանը չի բավարարվում, կստուգվի հաջորդ պայմանը՝ արդյոք եղանակը **Snow** է: Եվ եթե այս պայմանն էլ սխալ է, ծրագիրը կանցնի հաջորդ պայմանին՝ ստուգելու, արդյոք temp-ը փոքր է 10-ից: Քանի որ այս պայմանը ճիշտ է, կտպվի՝ **"Dress warmly"**:

Այս դեպքում, ցանկացած պայմանի ճիշտ լինելու դեպքում՝ հաջորդ պայմանները պարզապես չեն ստուգվի: Միայն այն դեպքում, երբ բոլոր պայմանները սխալ են, կաշխատեն else-ին վերագրված հրամանները:

Elif- գործողությունը կարելի է կատարել նաև օգտագործելով else և if սակայն այս դեպքում

ավելի է բարդանում ծրագրի տեսքը և ընթեռնելիությունը:

Վերոնշյալ ծրագիրը elif-ի կիրառմամբ

Հաճախ անհրաժեշտ է միաժամանակ ստուգել երկու պայման որևէ գործողություն կատարելու համար, և դա հնարավոր է իրականացնել մեկ if հրամանի միջոցով՝ պայմանները միավորելով:

Օրինակ՝ երբ անհրաժեշտ է ստուգել, արդյոք թիվը պատկանում է որոշակի միջակայքի, թե ոչ: Ստորև ստանում ենք թիվ, և եթե այն պատկանում է [4, 26) միջակայքին, ծրագիրը կտպի

"inside", հակառակ դեպքում՝ "outside":

Python ծրագրավորման լեզվում հնարավոր է շատ հեշտությամբ ստուգել, արդյոք թիվը պատկանում է որոշակի միջակայքի, թե ոչ: Ստորև ներկայացված է այդ ստուգման պարզ և հակիրճ ձևը.

Նման ձևով հնարավոր է կիրառել նաև բոլոր համեմատման նշանները՝ ինչպես փոքր կամ

հավասար ($\leq$), մեծ կամ հավասար ($\geq$), հավասար չէ ($\neq$) և այլն:

Ծրագրավորման մեջ հնարավոր է ստոգել պայմանների միաժամանակ ճիշտ լինելը գործողություն կատարելու համար կամ դրանցից որևէ մեկի ճիշտ լինելը դա կատարվում է $\text{and}()$ և $\text{or}()$ գործողությունների օգնությամբ որը հնարավորություն կտա կապել միմիանց

միջանի պայմաններ օրինակները կարող եք տեսնել ստորև:

Այս օրինակում ծրագիրը կտալի "Good day" միայն այն դեպքում, երբ ջերմաստիճանը լինի 18-ից մեծ, և եղանակը՝ "Sunny": Եթե այդ պայմաններից առնվազն մեկը սխալ է, ապա կաշխատի else հատվածը և կտալի "Unfavorable day":

Ստորև բերված օրինակում պայմանները կապվում են or օպերատորի օգնությամբ, և անգամ դրանցից միայն մեկի ճիշտ լինելու դեպքում ծրագիրը կտալի "Good day": Միայն այն դեպքում, երբ երկու պայմաններն էլ սխալ են, ծրագիրը կտալի "Unfavorable day":

Ի դեպ, կարելի է խմբավորել and և or պայմանները՝ ավելի բարդ խնդիրներ լուծելու համար:

Նշենք նաև, որ and և or օպերատորները հնարավոր է կիրառել ոչ միայն քանակական սահմանափակումների դեպքում, այլև ցանկացած տրամաբանական պայմանների միջև: Ստորև ներկայացված է մի օրինակ:

Այս դեպքում "Let's go out!" կտպվի, եթե օրն արձակուրդ է կամ եղանակը արևոտ է և ջերմաստիճանը մեծ է 18-ից: Բոլոր մնացած դեպքերում կտպվի "Better stay home.":

Ստորև կարող եք տեսնել՝ ինչպես է կայացվում որոշումը and և or օպերատորների կիրառման դեպքում՝ կախված պայմաններից:

Պայման 1	Պայման 2	Պայման 1 and Պայման 2	Պայման 1 or Պայման 2
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Ճրագրավորման մեջ երբեմն անհրաժեշտ է կատարել պայմանների բացասում, որը Python-ում իրականացվում է not օպերատորի օգնությամբ: not օպերատորը փոխում է պայմանի վերջնական արդյունքը՝ եթե այն True էր, կդառնա False, իսկ եթե False էր, կդառնա True:

Այստեղ is_sunny փոփոխականը սկզբում ունի արժեք True: Երբ օգտագործում ենք not is_sunny, այն կդառնա False: Հետևաբար, ծրագիրը կտպի It's sunny today., քանի որ not True դարձնում է False: Եթե is_sunny-ը լիներ False, ապա not-ը այն կդարձներ True և կկատարվեր առաջին կետը՝ It's not sunny today. :

Python- ում պայմանի փոխարեն կարելի է նշել նաև փոփոխականներ ստորև նշված աղյուսակում նշված է թե՛ կախված տիպից ինչ արժեքի դեպքում պայմանը կընդունի True և ինչ պայմանի դեպքում կընդունի False:

Տիպ	True արժեք	False արժեք
bool	True	False
int	Երբ թիվը տարբեր է 0-ից	Երբ թիվը 0 է
float	Երբ թիվը տարբեր է 0.0-ից	Երբ թիվը 0.0 է
str	Երբ տողը դատարկ չէ	Երբ տողը դատարկ է ("")
list	Երբ ցուցակը դատարկ չէ	Երբ ցուցակը դատարկ է ([])
tuple	Երբ tuples-ը դատարկ չէ	Երբ tuple-ը դատարկ է (())

dict	Երբ բառարանը դատարկ չէ	Երբ բառարանը դատարկ է ({})
set	Երբ set-ը դատարկ չէ	Երբ set-ը դատարկ է (set())

Առաջադրանքներ

- Գրել ծրագիր, որը մուտքում ստանում է ամբողջ թիվ տպում է ` այն դրական է, բացասական, թե՞ զրո:
- Գրել ծրագիր, որը մուտքում ստանում է ամբողջ թիվ ասում է` այն զույգ է, թե` կենտ:(օգտագործեք մնացորդով բաժանումը ստուգելու համար)
- Գրել ծրագիր, որը մուտքում կստանա մարդու տարիքը և կտաի՝ նա չափահաս է (≥ 18) թե անչափահաս:
- Գրել ծրագիր, որը մուտքում կստանա գաղտնաբառ և կստուգի՝ արդյոք այն համընկնում է նախապես որոշված գաղտնաբառի հետ (օրինակ՝ secret123):Եթե այո ապա կտաի yes հակառակ դեպքում no:
- Գրել ծրագիր, որը կստանա եռանկյան երեք կողմերը որպես մուտքային տվյալ և կստուգի՝ հնարավոր է այդ կողմերով եռանկյուն կազմել, թե ոչ:(խնդիրը լուծելիս օգտագործեք and օպերատորը)
- Գրել ծրագիր, որը կստանա երկու ամբողջ թիվ և կտաի մեծագույնը: Եթե երկուսն էլ հավասար են, ցուցադրել համապատասխան հաղորդագրություն:
- Գրել ծրագիր, որը մուտքում կստանա 0-ից 100 միջակայքում միավոր և կտաի համապատասխան գնահատականը՝ ըստ պայմանների (օր. $\geq 90 \rightarrow "A"$, $\geq 80 \rightarrow "B"$, և այլն):
- Գրել ծրագիր, որը կստանա օգտանուն և գաղտնաբառ և կստուգի՝ արդյոք մուտքային տվյալները ճիշտ են (օրինակ՝ օգտանունը admin, գաղտնաբառը 1234):
- Գրել ծրագիր, որը կստանա ամիսը (1-ից 12 միջակայքում) որպես ամբողջ թիվ և կտաի, թե որ սեզոնին է համապատասխանում տվյալ ամիսը:
- Գրել ծրագիր, որը կստանա մի թիվ և կստուգի՝ արդյոք այն պատկանում է [10, 20) միջակայքին:

11. Գրել ծրագիր, որը կստանա ջերմաստիճանը (թիվ) և եղանակը (օրինակ՝ "Sunny") և կտա "Good day", եթե ջերմաստիճանը >18 և եղանակը արևոտ է:
12. Գրել ծրագիր, որը մուտքում կստանա լույսի գույնը (Red, Yellow, Green) և կտա համապատասխան գործողությունը (օրինակ՝ "Stop", "Wait", "Go"):
13. Գրել ծրագիր, որը կստանա տարիքը և անվտանգության կոդը: Թույլատրել մուտք միայն եթե տարիքը ≥ 18 և կոդը ճիշտ է: (կոդը generationAi)
14. Գրել ծրագիր, որը կստանա մուտքային տվյալ (օրինակ՝ տրամաբանական փոփոխական կամ թիվ) և կօգտագործի not օպերատորը՝ փոխելու արդյունքը:
15. Գրել ծրագիր որը մուտքում կստանա տառ և կտա այն՝ եթե այն բազմապատկեր է:

Թեման 6. Set, Tuple (հավաքածուներ)

Մինչ Python լեզվով Tuple և Set կառույցներին անդրադառնալը, դիտարկենք դրանց գաղափարական արմատները իրական կյանքում և պատմական օգտագործման օրինակները:

Tuple-ի գաղափարը

Պատմության մեջ բազմաթիվ փաստաթղթեր ու իրավական գրառումներ ներկայացվել են որպես հաստատուն, անփոփոխվող արժեքների շարքեր՝ որոնք նախատեսված չեն փոփոխման համար: Սրանք հենց այնպիսի տվյալների շարքեր են, որոնք պահվում են Python-ի Tuple կառուցվածքներում:

Օրինակներ՝

- Օրենքների շարադրանքներ՝ օրինակ՝ Համուրաբիի օրենսգիրքը կամ Սահմանադրության հոդվածները: Դրանք գրված են հաստատուն ձևով, և փոփոխվում են միայն հատուկ գործընթացով, ոչ ծրագրի ընթացում:
- Աշխարհագրական կոորդինատներ (լայնություն, երկայնություն) նույնպես tuple-ի օրինակ է քանի որ յուրաքանչյուր կետի համար դա հաստատուն է և ենթակա չէ փոփոխման

Set-ի գաղափարը

Մարդկային մտածողության մեջ վաղուց է ձևավորվել գաղափարը, որ որոշ արժեքներ կարող են հայտնվել միայն մեկ անգամ: Սրան համարժեք է Python-ի Set կառուցվածքը:

Օրինակներ՝

- սոցիալական քարտի համարը կամ անձնագրային տվյալները՝ յուրահատուկ են յուրաքանչյուր անձի համար և չեն կարող կրկնվել:

- Հարկատուների կամ ընտրողների ցուցակներ՝ սահմանված են մեկական ներկայությամբ:

Tuple

Tuple-ը Python լեզվում տվյալների չփոփոխվող (*immutable*) հավաքածու է: Այն պարունակում է մի շարք տարրեր, որոնք տրվում են ստեղծման պահին և այլևս չեն կարող փոփոխվել: Tuple-ի ստեղծումը նման է ցուցակների (list) ստեղծմանը, սակայն [] փակագծերի փոխարեն օգտագործվում են () փակագծեր: Այն կարող է պարունակել տարբեր տիպի տարրեր՝ ինչպեսիք են թվեր, տողեր կամ բուլյան արժեքներ:

Tuple-ի հետ կարելի է կիրառել գրեթե նույն գործողությունները, ինչ ցուցակների (list) դեպքում: Բացառություն են կազմում միայն այն գործողությունները, որոնք կապված են տարրերի փոփոխման կամ ավելացման հետ: Քանի որ tuple-ը չփոփոխվող (*immutable*) կառուցվածք է, դրա վրա անթույլատրելի են հետևյալ մեթոդները՝ `.append()`, `.remove()`, `.pop()` ինչպես նաև տարրի փոփոխումը ըստ ինդեքսի (`t[i] = նոր_արժեք`):

Այս դեպքում կառաջանա սխալ

```
===== RESTART: C:/Users/GenAi / Test.py =====
Traceback (most recent call last):
  File "C:/Users/Gor Hovsepyan/Documents/1.py", line 2, in <module>
    my_tuple[0] = 3
TypeError: 'tuple' object does not support item assignment
```

Ստորև ներկայացված են բոլոր այն գործողությունները, որոնք հասանելի են tuple-ների հետ աշխատելիս

Գործողություն / Մեթոդ	Բացատրություն	Օրինակ	Արդյունք
<code>t = (1, 2, 3)</code>	Ստեղծում է նոր tuple երեք տարրով	<code>t = (1, 2, 3)</code>	<code>(1, 2, 3)</code>
<code>t[i]</code>	Վերադարձնում է i ինդեքսով տարրը	<code>t[1]</code>	2
<code>t[i:j]</code>	Կտրտում է tuple-ը i-ից մինչև j-1-ը	<code>t[0:2]</code>	<code>(1, 2)</code>
<code>len(t)</code>	Վերադարձնում է tuple-ի երկարությունը	<code>len(t)</code>	3

x in t	Ստուգում է՝ արդյոք x տարրը առկա է tuple-ում	2 in t	True
t.count(x)	Հաշվում է x տարրի քանակը tuple-ում	t.count(2)	1
t.index(x)	Վերադարձնում է x տարրի առաջին հանդիպման ինդեքսը	t.index(3)	2
tuple([x, y])	Ցուցակը դարձնում է tuple	tuple([4, 5])	(4, 5)
tuple("abc")	Տողը դարձնում է tuple՝ որպես սիմվոլների հաջորդականություն	tuple("abc")	('a', 'b', 'c')
tuple(sorted(t))	Վերադարձնում է տեսակավորված նոր tuple	tuple(sorted((3, 1, 2)))	(1, 2, 3)
min(t) / max(t)	Վերադարձնում է ամենափոքր / ամենամեծ տարրը	min(t) / max(t)	1 / 3
sum(t)	Վերադարձնում է թվային տարրերի գումարը	sum((1, 2, 3))	6
tuple1 + tuple2	Միավորում է երկու tuple	(1, 2) + (3, 4)	(1, 2, 3, 4)
tuple1 * n	Կրկնում է tuple-ի պարունակությունը n անգամ	(1, 2) * 2	(1, 2, 1, 2)
x, y = (5, 7)	Տվյալների փաթեթավորում / ապափաթեթավորում՝ փոփոխականների միջոցով	x, y = (5, 7)	x = 5, y = 7
t = (1,)	Ստեղծում է մեկ տարրով tuple (հարկավոր է ստորակետ)	type((1,))	<class 'tuple'>
t = ()	Ստեղծում է դատարկ tuple	t = ()	()

Ինչպես նաև անհրաժեշտ է նշել որ tuple-ի համար չեն գործում այն օպերատորները և մեթոդները որոնք կատարում են վերադասավորում օրինակ `sort()` , `reverse()`: Սակայն երբեմն անհրաժեշտ է լինում կատարել նմանատիպ գործողություններ. օրինակ՝ երբ հարկավոր է շրջել tuple-ը: Այս դեպքում կարելի է այն փոխակերպել ցուցակի, կատարել անհրաժեշտ փոփոխությունները և այնուհետև կրկին ձևափոխել tuple-ի: Ստորև բերված է օրինակ թե ինչպես կարելի է ավելացնել նոր տարր:

Tuple-ը նաև հնարավորություն է տալիս փոխանակել (տեղերով փոխել) փոփոխականների արժեքները: Դրա օրինակն բերված է ստորև.

Ի՞նչ է տեղի ունենում այստեղ.

1. Փաստորեն, արտահայտությունը $a, b = b, a$ ստեղծում է թաքնված tuple` $a, b = (b, a)$:
2. Այնուհետև տեղի է ունենում ապափաթեթավորում (unpacking)` նոր արժեքները վերագրվում են համապատասխան փոփոխականներին:

Սակայն այս դեպքում անհրաժեշտ է հիշել որ փոփոխականների քանակը երկու կողմերում անհրաժեշտ է որպեսզի լինեն հավասար հակառակ դեպքում կառաջանա սխալ(Error):

Set

Set-ը Python-ում տվյալների փոփոխվող, չկարգավորված կառուցվածք է, որն ունի մի կարևոր առանձնահատկություն. այն չի թույլատրում կրկնվող տարրեր: Այսինքն` յուրաքանչյուր տարր կարող է գոյություն ունենալ միայն մեկ անգամ:

Set-ը որոշ առումներով նման է ցուցակին (list). այն կարող է պարունակել տարբեր տիպի տարրեր (թվեր, տողեր, բուլյան արժեքներ) և կարող է փոփոխվել` տարրեր ավելացնելով կամ հեռացնելով:

Սակայն գլխավոր տարբերությունը ցուցակի և set-ի միջև հետևյալն է.

Set-ում կրկնվող տարրերը ավտոմատ կերպով հանվում են թողնելով կրկնվողներից միայն մեկը:

Սա շատ կարևոր հատկություն է, երբ աշխատում ենք այնպիսի տվյալների հետ, որոնք պետք է լինեն յուրահատուկ: Օրինակ`

- Մարդուն տարբերակող տվյալներ (սոցիալական քարտի համարը, անձնագրի սերիա)
- Հարկատուների կամ ընտրողների ցանկեր
- Բազմությունների վրա գործողություններ` միավորում, հատում, տարբերություն

Set-ը ստեղծվում է ցուցակների նման ուղակի [] փակագծերի փոխարեն օգտագործվում է {}:

Set-ում տարրերի ավելացումը տարբերվում է ցուցակներում տարրերի ավելացումից. եթե ցուցակներում օգտագործվում է append() հրամանը, ապա այստեղ օգտագործվում է add() հրամանը: Օրինակ՝
Արդյունք՝

```
===== RESTART: C:/I
{0, 1, 2, 5, 7, 8}
{0, 1, 2, 4, 5, 7, 8}
|
```

Այս օրինակում կարող ենք նաև նկատել, որ set-ը չի պահպանում տարրերի ավելացման հերթականությունը, և այդ կարգը կարող է փոփոխվել: Նույն իրավիճակն առաջանում է նաև այն դեպքում, երբ ցուցակը (list) վերածում ենք set-ի՝ կրկնվող տարրերը հանելու նպատակով: **Ցուցակը set-ի ձևափոխելը նման է մեկ փոփոխականի տիպից մյուսի վերափոխմանը:** Python-ում տվյալների կառուցվածքներն իրար կարելի է փոխարկել՝ օգտագործելով համապատասխան կոնստրուկտորներ (list(), set(), tuple() և այլն): Ցուցակը set դարձնելը կատարվում է հետևյալ կերպ՝

```
===== RESTART
{0.7, 1, 2, 0, 5, 18}
```

Արդյունք՝ , Այստեղ կարող ենք տեսնել որ թվերը set-ում դասավորված են խառը իսկ կրկնվող տարրերը հեռացվել են:
Կարևոր է հիշել, որ ինչպես ցուցակների (list), այնպես էլ set-ների դեպքում, երբ ստեղծում ենք նոր փոփոխական set2 և այն հավասարեցնում ենք set1-ին, ապա իրականում set2-ը չի դառնում անկախ կրկնօրինակ:
Փոխարենը, երկու անունն էլ (set1, set2) սկսում են հղվել նույն set-ին, այսինքն՝ հիշողության նույն հասցեին:

Սա նշանակում է, որ եթե որևէ փոփոխություն կատարվի դրանցից որևէ մեկի վրա, ապա փոփոխությունն անմիջապես կանդրադառնա երկուսին էլ:

Միասլ օրինակի օրինակ՝

```
===== 1
{1, 2, 3, 4}
```

Արդյունք՝ **{1, 2, 3, 4}** Չնայած փոփոխությունը կատարվել է միայն set1-ի վրա, այն ազդել է նաև set2-ի վրա, քանի որ դրանք երկուսն էլ հղվում են նույն set-ին:

Եթե ցանկանում ենք ունենալ անկախ կրկնօրինակ

Այդ դեպքում անհրաժեշտ է օգտագործել copy() մեթոդը, որը ստեղծում է set-ի նոր օրինակ՝ նոր հիշողության հասցեում:

```
===== REST1
set1- {1, 2, 3, 4}
set2- {1, 2, 3}
```

Արդյունք՝ **{1, 2, 3, 4}** Այստեղ set1-ի փոփոխությունը **չի ազդում** set2-ի վրա:

Ստորև բերված են բոլոր \$ունկցիաները և հրամանները, որոնք նախատեսված են set-ների հետ աշխատելու համար: Անհրաժեշտ է հիշել ստորև բերված օրինակներում օգտագործվում է $s = \{1,2,3\}$ $s1 = \{1,2,3\}$ set-երը:

Հրաման / Մեթոդ	Բացատրություն	Օրինակ	Արդյունք
s = set()	Ստեղծում է դատարկ set	s = set()	set()
s = {1, 2, 3}	Ստեղծում է set՝ նախապես նշված տարրերով	s = {1, 2, 3}	{1, 2, 3}
s.add(x)	Ավելացնում է x տարրը set-ի մեջ (եթե արդեն չկա)	s.add(4)	{1, 2, 3, 4}

s.remove(x)	Հեռացնում է x տարրը: Եթե չկա՝ առաջանում է սխալ(Error)	s.remove(2)	{1, 3, 4}
s.discard(x)	Հեռացնում է x տարրը: Եթե չկա՝ ոչինչ չի լինում	s.discard(10)	{1, 3, 4}
s.pop()	Հեռացնում է պատահական տարր և վերադարձնում այն	x = s.pop()	x = 1, s = {3, 4} (օրինակ)
s.clear()	Հեռացնում է բոլոր տարրերը set-ից	s.clear()	set()
len(s)	Վերադարձնում է տարրերի քանակը	len(s)	3
x in s	Ստուգում է՝ արդյոք x տարրը կա set-ում վերադարձնում է True եթե կա False եթե չկա	2 in s	True
s1.union(s2)	Վերադարձնում է նոր set՝ երկու set-երի միավորմամբ	s2 = {4,5,6} s1.union(s2) print(s1)	{1,2,3,4,5,6}
s1.intersection(s2) կամ s1 & s2	Վերադարձնում է նոր set՝ երկու set-ների հատմամբ	{1, 2, 3} & {2, 3, 4}	{2, 3}
s1.difference(s2) կամ s1 - s2	Վերադարձնում է այն տարրերը, որոնք կան s1-ում, բայց չկան s2-ում	{1, 2, 3} - {2, 3}	{1}
s1.symmetric_difference(s2) կամ s1 ^ s2	Տարբերություն առանց հատման՝ վերադարձնում է միայն յուրահատուկ տարրերը	{1, 2, 3} ^ {2, 4}	{1, 3, 4}
s.update(s2)	Միավորում է s2-ի տարրերը s-ի հետ տեղում (in-place)	s.update({10, 20})	s = {1, 3, 4, 10, 20}
s.copy()	Վերադարձնում է set-ի պատճենը	s2 = s.copy()	s2 = {1, 2, 3}
set(list)	Ցուցակից ստեղծում է set (կրկնվող տարրերը հանում է)	set([1, 2, 2, 3])	{1, 2, 3}
sorted(set)	Տեսակավորում է set-ը և վերադարձնում ցուցակ	sorted({3, 1, 2})	[1, 2, 3]

Եկեք ավելի մանրամասն ուսումնասիրենք վերոնշյալ հրամաններից 3-ը՝ intersection, difference և symmetric_difference:

Intersection

Intersection գործողությունը թույլ է տալիս գտնել երկու set-երի միջև եղած հատումը, այսինքն՝ այն տարրերը, որոնք առկա են և՛ առաջին, և՛ երկրորդ set-ում:

Intersection գործողությունը ունի նաև կրճատ գրելու ձև՝ & նշանի օգնությամբ:

Այսինքն՝ հետևյալ երկու գրառումներն էլ հավասարաթեք են.

```
=====
{3, 4}
```

Արդյունք՝ Ստացանք նոր set որտեղ երկու set-երի հատումն է:

Difference

Difference մեթոդը թույլ է տալիս առանձնացնել այն տարրերը, որոնք առկա են առաջին set-ում և բացակայում են երկրորդում: Կրճատ գրելու ձևը - (հանման գործողություն) նշանն է:
Այս գործողությունը ցույց է տալիս երկու բազմությունների միջև միակողմանի տարբերությունը:

```
=====
{1, 2}
{1, 2}
```

Արդյունքը՝ Այս դեպքում ստանում ենք նոր set որտեղ միայն այն տարրերն են որոնք կան առաջին set-ում և բացակայում են երկրորդում:

symmetric_difference

symmetric_difference մեթոդը հնարավորություն է տալիս ստանալ այն տարրերը, որոնք առկա են միայն առաջին կամ միայն երկրորդ set-ում, բայց ոչ երկուսում միաժամանակ:
Այլ կերպ ասած՝ վերադարձնում է տարբերությունը՝ առանց հատման: Կրճատ գրելաձևը ^ նշանն է:

Արդյունք՝

```
{1, 2, 5, 6}
{1, 2, 5, 6}
```

Ցուցակ set tuple հիմնական տարբերություն

Հատկություն	List (Ցուցակ)	Set (Բազմություն)	Tuple (Թափլ)
Փակագծերի տեսակը	[]	{}	()
Փոփոխելիություն (mutable)	Այո	Այո	Ոչ
Կրկնվող տարրերի թույլատրություն	Այո	Ոչ	Այո
Տարրերի հերթականության պահպանում	Այո	Ոչ	Այո
Տարրերի տեսակները	Ցանկացած	Ցանկացած (բացի փոփոխելի օբյեկտներից)	Ցանկացած
Օգտագործման բնույթ	Տվյալների հերթական հավաքածու, որը հաճախ փոփոխվում է	Եզակի տարրերով հավաքածու, առանց կրկնության	Տվյալների ֆիքսված (չփոփոխվող) հավաքածու

Առաջադրանքներ

1. Ստեղծիր tuple, որը պարունակում է երեք տարր՝ ամբողջ թիվ, տող և բուլեան արժեք:
2. Տպիր երկրորդ տարրը տրված tuple-ից:
3. Փորձիր փոխել tuple-ի որևէ տարրը և նկարագրիր ստացված սխալը:
4. Կատարիր փոփոխականների արժեքների փոխանակում tuple-ի միջոցով:
5. Ստեղծիր ցուցակ, փոխակերպիր այն tuple-ի և հակառակը:
6. Ստեղծիր set, որը պարունակում է 5, 2, 3 թվերը:
7. Ավելացրու նոր տարր՝ օգտագործելով համապատասխան մեթոդը:
8. Փորձիր ավելացնել կրկնվող տարր և բացատրի արդյունքը:
9. Հեռացրու տարր remove() մեթոդով և փորձիր հեռացնել բացակայող տարր:
10. Հեռացրու տարր discard() մեթոդով, որն իրականում բացակայում է:
11. Ստեղծիր երկու set և գտիր դրանց intersection-ը:
12. Ստեղծիր երկու set և գտիր դրանց difference-ը:
13. Ստեղծիր երկու set առաջինը պարունակում է 1,4,7,3 թվերը երկրորդը ստեղծիր առաջինի կրկնորինակը և հեռացրու նրա մեջից 4-և այնուհետև ավելացրու 0,12,6 և գտիր դրանց symmetric_difference-ը:
14. Ստեղծիր ցուցակ կրկնվող տարրերով և այն վերածիր set-ի:
15. Ստուգիր in օպերատորով՝ արդյոք որևէ տարր առկա է set-ում:
16. Փորձիր նաև մյուս հրամանների աշխատանքը

Թեմա 7. Բառարան (Dictionary)

Շատ վաղ ժամանակներից մարդը ձգտել է պահպանել և փոխանցել լեզվական գիտելիքը հաջորդ սերունդներին: Այս ցանկությունը հիմք դրեց բառարանների ստեղծմանը: Թեև մենք այսօր բառարանը պատկերացնում ենք որպես գրադարանային հաստ գիրք կամ էլեկտրոնային հավելված, նրա արմատները գնում են հազարավոր տարիներ հետ: Երեք հազար տարի առաջ, Միջագետքի հողերում, սումերներն ու աքքադացիները սկսեցին կազմել կավե սալիկների վրա գրված բառարաններ: Այս սալիկները պարունակում էին սումերերեն բառեր՝ աքքադերեն թարգմանություններով: Դրանք դասավորված էին թեմատիկ խմբերով՝ կենդանիներ, բույսեր, աշխատանքներ: Այս հնագույն բառարանները ծառայում էին որպես ուսուցման և թարգմանության գործիք հնագույն դպրոցներում:

Ժամանակի ընթացքում բառարանային միտքը տարածվեց Հունաստանում, Հռոմում, ապա՝ միջնադարյան Եվրոպայում: 1604 թվականին անգլիացի քահանա Ռոբերտ Զոուլիդին հրատարակեց առաջին միալեզու անգլերեն բառարանը՝ "Table Alphabeticall" վերնագրով: Այն ներառում էր մոտ 2500 բառ, որոնց բացատրությունները ուղղված էին հասարակ մարդկանց՝ «դժվար ու անհասկանալի բառերից խուսափելու» նպատակով:

Python ծրագրավորման լեզվում բառարանները (dictionary) կատարում են նույն դերը, ինչ իրական կյանքի լեզվաբանական բառարանները: Դրանք նախատեսված են բանալի-արժեք (key-value) զույգեր պահելու համար:

Դա կարելի է համեմատել անգլերեն-հայերեն բառարանի հետ.

- Բանալի բառը (key)՝ անգլերեն բառն է,

- Արժեքը (value)՝ նրա հայերեն թարգմանությունը կամ բացատրությունը:

Այս համեմատությունից պարզ է դառնում, որ՝

1. յուրաքանչյուր բանալի եզակի է, քանի որ իրական բառարանում մեկ բառ ունի միայն մեկ ընդգրկված բացատրություն:
2. Եթե փորձենք կրկնել նույն բանալին, ապա Python-ում այն կփոխարինի նախորդը, այլ ոչ թե կրկնվի:

Ստորև կարող ենք տեսնել թե՛ ինչպես է ստեղծվում բառարանը python-ում:

Python-ում բառարանների բանալիներն ու արժեքները կարող են լինել գրեթե ցանկացած տվյալների տիպի: Սակայն պարտադիր չէ, որ բանալիները և արժեքները լինեն նույն տիպի:

Կարող ենք, օրինակ, ունենալ ամբողջ թվային բանալի և տեքստային արժեք, կամ հակառակը:

Python-ում բառարան ստեղծելու համար գոյություն ունեն երկու հիմնական ձև.

1. Օգտագործելով {} փակագծերը, որտեղ տվյալները ներմուծվում են բանալի:արժեք (key: value) զույգերով, միմյանցից բաժանված ստորակետերով:
2. Օգտագործելով dict() ֆունկցիան, երբ բանալիները տրվում են որպես արգումենտ՝ առանց փակագծերի:

:

Արդյունք՝

```
===== RESTART: C:\Users\GenAi\Test.py ==  
{'name': 'Anna', 'age': 17, 'city': 'Yerevan'}  
{'name': 'Anna', 'age': 17, 'city': 'Yerevan'}
```

Բառարաններում, երբ ցանկանում ենք ստանալ որևէ տարր, ի տարբերություն ցուցակների, օգտագործում ենք ոչ թե ինդեքս (համար), այլ բանալի: Օրինակ՝ եթե ցանկանում ենք ստանալ այն արժեքը, որը համապատասխանում է 'name' բանալուն, ապա պետք է փակագծերում նշենք 'name', ոչ թե թիվ, ինչպես ցուցակների դեպքում:

Արդյունք՝

```
=====:  
Anna  
>>> |
```

Բացի առանձին տարրը ստանալուց ունենք նաև հնարավորություն ստանալու բոլոր բանալիները(key) և բոլոր արժեքները(value) առանձին ցուցակների տեսքով դրա համար օգտագործում ենք .keys() և .values() հրամանները:

Արդյունքում ստանում ենք հատուկ տեսակի ցուցակներ, որոնց հետ արդեն կարող ենք առանձին աշխատել՝ թերթել, փոփոխել կամ վերածել սովորական ցուցակի: Սակայն

անհրաժեշտ է հիշել, որ այդ ցուցակների վրա կատարված փոփոխությունները չեն ազդում սկզբնավոր բառարանի վրա:

```
===== RESTART: C:\Users\Gei
dict_keys(['name', 'age', 'city'])
dict_values(['Anna', 17, 'Yerevan'])
>>>
```

Ինչպես ցուցակների դեպքում կարող էինք for ցիկլի միջոցով հերթով ստանալ բոլոր տարրերը, նույնը հնարավոր է անել նաև բառարանների (dictionary) հետ աշխատելիս:

Python-ում կարող ենք թերթել բառարանի պարունակությունը երեք հիմնական ձևով.

1. Ստանալ միայն բանալիները (keys)

Արդյունք՝

```
=====
name
age
city
>>>
```

Սա նույնն է, ինչ `for key in my_dict.keys()`

2. Ստանալ միայն արժեքները (values)

Արդյունք՝

```

=====
Anna
17
Yerevan
>>> |

```

3. Ստանալ միաժամանակ և՛ բանալին, և՛ արժեքը (items)

Սա ամենահաճախ օգտագործվող տարրերակն է, երբ պետք է զույգերով (key-value) մշակել բառարանի պարունակությունը:

Բառարանում տարրերի ավելացում

Python-ում բառարանին նոր տարր ավելացնելը շատ պարզ է: Պետք է պարզապես նշել նոր բանալի և նրան վերագրել համապատասխան արժեք:

```

===== RESTART: C:\Users\GenAi\Test.py :
{'name': 'Anna', 'age': 17, 'city': 'Yerevan'}
>>> |

```

Արդյունք՝

Կարևոր է հիշել որ եթե այդ բանալին արդեն գոյություն ունի բառարանում, ապա նրա արժեքը կփոխարինվի նորով. Օրինակ՝

```
===== RESTART: C:\>
{'name': 'Anna', 'age': 20}
>>>
```

Արդյունք՝

Բացի սովորական dict[key] = value եղանակից, կարելի է միաժամանակ մի քանի տարր ավելացնել կամ փոփոխել՝ օգտագործելով update() մեթոդը:

Այս մեթոդը ընդունում է՝

- մի այլ բառարան
- կամ բանալիներ՝ որպես արգումենտներ (key=value տեսքով)

1. Օրինակ՝ այլ բառարանի միջոցով

Արդյունք՝

```
===== RESTART: C:\Users\GenAI\Test.py =====
{'name': 'Anna', 'age': 17, 'city': 'Yerevan', 'email': 'anna@mail.com'}
>>>
```

2. Օրինակ՝ բանալի=արժեք սինտաքսով

```
===== RESTART: C:\Users\GenAI\Test.py =====
{'name': 'Anna', 'age': 18, 'phone': '077123456'}
>>>
```

Արդյունք՝

Բառարանների հետ աշխատանքի համար նախատեսված հիմնական մեթոդները և կիրառմանները

Ստորև բերված են բոլոր հիմնական կիրառմանները և մեթոդները, որոնք օգտագործվում են Python լեզվում բառարանների հետ աշխատելիս:

Բոլոր օրինակները հիմնված են հետևյալ նախնական բառարանի վրա

Հրաման / Մեթոդ	Բացատրություն	Օրինակ	Արդյունք
<code>d["name"]</code>	Ստանում է նշված բանալուն համապատասխան արժեքը:	<code>d["name"]</code>	"Anna"
<code>d.get("email")</code>	Ապահով կերպով ստանում է արժեքը՝ եթե բանալին չկա, վերադարձնում է None:	<code>d.get("email")</code>	None
<code>d["age"] = 18</code>	Փոխում է 'age' բանալու արժեքը:	<code>d["age"] = 18</code>	{'name': 'Anna', 'age': 18, 'city': 'Yerevan'}
<code>d.update({"city": "Gyumri", "phone": "077"})</code>	Միաժամանակ ավելացնում կամ թարմացնում է մի քանի տարր:	<code>d.update({"city": "Gyumri", "phone": "077"})</code>	{'name': 'Anna', 'age': 17, 'city': 'Gyumri', 'phone': '077'}
<code>d.update(email="a@mail.com")</code>	Ավելացնում է 'email' բանալի՝ համապատասխան արժեքով:	<code>d.update(email="a@mail.com")</code>	{'name': 'Anna', 'age': 17, 'city': 'Yerevan', 'email': 'a@mail.com'}
<code>d.keys()</code>	Վերադարձնում է բոլոր բանալիները:	<code>d.keys()</code>	dict_keys(['name', 'age', 'city'])
<code>d.values()</code>	Վերադարձնում է բոլոր արժեքները:	<code>d.values()</code>	dict_values(['Anna', 17, 'Yerevan'])
<code>d.items()</code>	Վերադարձնում է բոլոր բանալի-արժեք զույգերը:	<code>d.items()</code>	dict_items([('name', 'Anna'), ('age', 17), ('city', 'Yerevan')])
<code>d.pop("name")</code>	Հեռացնում է 'name' բանալին և վերադարձնում նրա արժեքը:	<code>d.pop("name")</code>	'Anna' և {'age': 17, 'city': 'Yerevan'}
<code>d.popitem()</code>	Հեռացնում է վերջին ավելացված բանալի-արժեք	<code>d.popitem()</code>	Օրինակ՝ ('city', 'Yerevan')

	զույգը:		
d.clear()	Դատարկում է բառարանը:	d.clear()	{}
"name" in d	Ստուգում է՝ արդյոք 'name' բանալին առկա է:	"name" in d	True կամ False (կախված d-ից)
del d["age"]	Չեռացնում է 'age' բանալին:	del d["age"]	{'name': 'Anna', 'city': 'Yerevan'}
d.copy()	Ստեղծում է բառարանի պատճեն:	d2 = d.copy()	d2 = {'name': 'Anna', 'age': 17, 'city': 'Yerevan'}
len(d)	Վերադարձնում է բառարանի երկարությունը (զույգերի քանակը):	len(d)	3

Կարևոր է հիշել, որ ինչպես ցուցակների (list) կամ բազմությունների (set) դեպքում, եթե գրենք.

Արդյունք՝

```
===== RESTART: C:\Users\GenAi\ Test.py ===
{'name': 'Anna', 'age': 18, 'city': 'Yerevan'}
{'name': 'Anna', 'age': 18, 'city': 'Yerevan'}
>>>|
```

ապա մենք իրականում չենք ստեղծում նոր բառարան, այլ ուղղակի ստեղծում ենք նոր անուն (փոփոխական), որը հղվում է նույն (նախկին) բառարանի հիշողության հասցեին:

Այսինքն՝ d-ում կամ new_dict-ում կատարված ցանկացած փոփոխություն կանդրադառնա մյուսի վրա նույնպես:

Եթե ցանկանում ենք ստանալ բառարանի անկախ կրկնօրինակ, ապա պետք է օգտագործենք copy() մեթոդը.

Արդյունք՝

```
===== RESTART: C:\User
{'name': 'Anna', 'age': 17}
{'name': 'Anna', 'age': 18}
>>>
```

Առաջադրանքներ

1. Ստեղծիր բառարան, որը պարունակում է անձի անունը, ազգանունը և տարիք՝
2. Տալի բառարանից միայն 'name' և 'age' արժեքները:
3. Ստուգիր՝ արդյոք 'email' բանալին առկա է բառարանում:
4. Ավելացրու նոր բանալի 'city', որի արժեքը կլինի քո ցանկալի քաղաքը:
5. Փոխիր 'age' բանալու արժեքը՝ մեծացնելով այն 1-ով:
6. Ջնջիր 'surname' բանալին բառարանից և տալի արդյունքը:
7. Ստեղծիր երկու բառարան՝ անձի հիմնական տվյալներով և կոնտակտային տվյալներով:
Միավորիր դրանք update() մեթոդով:
8. Օգտագործելով keys() և values(), ստացիր բոլոր բանալիները և արժեքները առանձին ցուցակներում:
9. Կրկնօրինակիր բառարանը copy() մեթոդով և փոխիր պատճենում մեկ արժեք:
Համեմատիր սկզբնական և պատճենը:
10. Օգտագործիր for ցիկլ՝ տպելու բոլոր բանալի-արժեք զույգերը հետևյալ ձևաչափով՝

11. Ստեղծիր բառարան, որտեղ բանալիներն են առարկաների անուններ, իսկ արժեքները՝ համապատասխան գնահատականներ: Հաշվիր միջին միավորը:
12. Գեներացրու բառարան, որտեղ բանալիները թվերն են 1-ից 5, իսկ արժեքները՝ դրանց բառակուսիները:
13. Օգտագործողից ստացիր անուններ ու գնահատականներ և ստեղծիր բառարան: Տպիր բոլոր ուսանողներին, ում միավորը > 80 է:
14. Ստեղծիր բառարան, որը հաշվում է, թե քանի անգամ է յուրաքանչյուր տառ հանդիպում տրված բառում: Օրինակ՝
"banana" $\rightarrow$ {'b':1, 'a':3, 'n':2}
15. Տրված է երկու բառարան: Ստեղծիր երրորդը, որտեղ կլինեն միայն այն բանալի-արժեք զույգերը, որոնք ընդհանուր են երկու բառարաններում և արժեքները նույնպես համընկնում են:

Թեմա 8. Ֆունկցիաներ (Functions)

Մարդիկ իրենց ողջ կյանքի ընթացքում մշտապես փորձում են ավտոմատացնել մոնոտոն և կրկնվող գործողությունները: Օրինակ՝ գործարաններում հյուրի շշալցման գործընթացը նախկինում կատարվում էր մարդկանց կողմից, սակայն այսօր ստեղծվել են ավտոմատացված հոսքագծեր, որոնք ստանում են դատարկ շշերը, լցնում հյութով և առանց մարդկային միջամտության ապահովում են պատրաստի արտադրանքը:

Նույն գաղափարը գործում է ծրագրավորման մեջ: Ֆունկցիաները ստեղծվել են՝ օգնելու ծրագրավորողներին կազմակերպել կոդը, նվազեցնել կրկնությունը և հեշտացնել փոփոխությունները:

Ֆունկցիան իրենից ներկայացնում է ծրագրի մի հատված, որն ունենում է անուն: Ամեն անգամ, երբ մեզ անհրաժեշտ է այդ գործողությունը, մենք պարզապես կանչում ենք այդ ֆունկցիան՝ գրելով անունը, և ծրագիրը կատարում է այդ գործողությունները առանց ծրագրի կրկին գրելու անհրաժեշտության: Այսպիսով, ֆունկցիաները օգնում են ստեղծել մաքուր, ընթեռնելի և վերաօգտագործվող կոդ:

Ֆունկցիաների տեսակներ և ստեղծում

Python-ում ֆունկցիաները ստեղծվում են `def` բանալի բառի օգնությամբ, որից հետո տրվում է անուն (որին տրվում են նույն պահանջները, ինչ փոփոխականին), և փակագծերում նշվում են մուտքային արժեքները ընդունող փոփոխականները: Եթե մուտքային արժեք անհրաժեշտ չէ,

ապա փակագծերը թողնում ենք դատարկ: Այնուհետև դրվում է : նշանը, և հաջորդող տողը գրվում է մեկ Tab առաջ՝ նշելով այն ծրագիրը, որը պետք է կատարվի ֆունկցիան կանչելիս: ֆունկցիայի կառուցվածքը՝

Return հրամանը օգտագործվում է հենց արժեքը վերադարձնելու համար եթե ֆունկցիան արժեք չի վերադարձնում ապա return հրամանը չի գրվում:

ֆունկցիաները կարելի է բաժանել հիմնական 4 տեսակների.

1. Մուտքային արժեք ընդունող և արժեք չվերադարձնող
2. Մուտքային արժեք ընդունող և արժեք վերադարձնող
3. Մուտքային արժեք չընդունող և արժեք չվերադարձնող
4. Մուտքային արժեք չընդունող և արժեք վերադարձնող

Մուտքային արժեք ընդունող և արժեք չվերադարձնող

Այս տեսակի ֆունկցիաները ստանում են մուտքային տվյալներ, բայց ոչինչ չեն վերադարձնում: Դրանք հիմնականում օգտագործվում են ինչ-որ գործողություն կատարելու համար, օրինակ՝ տպել տեղեկատվություն:

Այսպիսի ֆունկցիայի օրինակ է նամակը ձևավորված կերպով տպող ֆունկցիան: Այն մուտքում ստանում է՝

- նամակի տեքստը,
- ուղարկողի անունը,
- ժամը,

և տպում է այն համապատասխան ձևաչափով:

Օրինակ՝

Այստեղ կարող ենք տեսնել նաև ֆունկցիայի կանչը: Ֆունկցիան գործարկելու համար նշվում է դրա անունը՝ այս դեպքում `print_message`, ապա փակագծերում տրվում են մուտքային արժեքները՝ նամակի տեքստը, ուղարկողի անունը և ժամը:

Այս արժեքները կարող են տրվել անմիջականորեն (օրինակ՝ ("Hi: meeting at 15:00.", "Ani", "14:30")) կամ փոփոխականների միջոցով: Կանչի երկրորդ տողում կարող ենք տեսնել, թե ինչպես ենք նույն ֆունկցիան օգտագործում փոփոխականներ փոխանցելով:

Այսպիսով՝ մենք ստանում ենք հնարավորություն մեկ անգամ գրելով տպելու գործողությունը, այն օգտագործել բազմաթիվ անգամներ՝ առանց կրկնօրինակման:

Կարևոր է հիշել.

- Ֆունկցիան կանչելիս մուտքային արժեքները պետք է տրվեն ճիշտ հերթականությամբ,

- Եվ ունենան պահանջվող տիպը:

Օրինակ՝ եթե ֆունկցիան կատարում է մաթեմատիկական գործողություններ, մուտքային արժեքները պետք է լինեն թվեր: Եթե փոխանցենք տեքստեր, ծրագիրը կսխալվի:

Արդյունք՝

```

===== RESTART: C:\Users\GenAi\Test.py =====
==== New message ====
Time: 14:30
Sender: Ani
Message-` Hi: meeting at 15:00.
=====

==== New message ====
Time: 15:35
Sender: Gor
Message-` Hi: meeting time changed to 19:00.
=====

>>> |

```

Մուտքային արժեք ընդունող և արժեք վերադարձնող

Այստեղ կարող ենք դիտարկել առակուսի հավասարումը $ax^2 + bx + c = 0$ լուծող ֆունկցիա, որը հաշվում է հավասարման լուծումների քանակը: Ֆունկցիան մուտքում ստանում է a , b և c գործակիցները և վերադարձնում է հնարավոր լուծումների քանակը՝ 0, 1 կամ 2:

Որոշ դիտարկումներից պարզ է դառնում, որ արժեք վերադարձնող ֆունկցիաները տարբերվում են նախորդ օրինակներից: Եթե նախորդ օրինակում (օրինակ՝ տպող ֆունկցիայի օրինակում) մենք պարզապես կանչում էինք ֆունկցիան՝ `print_message(...)`, ապա այս դեպքում ֆունկցիան կատարում է հաշվարկ և վերադարձնում է որոշակի արժեք:

Այս վերադարձված արժեքը, օրինակ՝ ամբողջ թիվ, տասնորդական կամ տեքստ, մենք պետք է կա՛մ տպենք, կա՛մ պահպանենք փոփոխականի մեջ, որպեսզի հետագայում կարողանանք օգտագործել: Եթե ուղղակի կանչենք ֆունկցիան առանց պահպանելու կամ տպելու, ապա արդյունքը կկորչի, քանի որ այն չպահպանվեց ոչ մի տեղ:

Կարևոր է հիշել, որ եթե ֆունկցիան որևէ պայմանով չի կատարում return, ապա այն լռելյայն վերադարձնում է None:

Ուստի ցանկալի է, որ ցանկացած դեպքում ֆունկցիան վերադարձնի ինչ-որ արժեք, ինչպես մեր օրինակում. նույնիսկ եթե արմատներ չկան, այն վերադարձնում է 0:

Արդյունք՝

```
=====
2
1
0
>>>|
```

Մուտքային արժեք չընդունող և արժեք չվերադարձնող

Այս տեսակի ֆունկցիաներ չեն ստանում որևէ մուտքային արժեք և չեն վերադարձնում արդյունք: Դրանք սովորաբար պարզապես կատարում են որոշ գործողություն՝ առանց որևէ տվյալ պահանջելու կամ վերադարձնելու:

Սովորաբար օգտագործվում են՝

- տեղեկություն տպելու,
- ողջույնի կամ ազդանշանային հաղորդագրության ցուցադրման նպատակով:

Օրինակ՝

Այս դեպքում ֆունկցիայի կանչը կատարվում է նույն կերպ, ինչպես մուտքային արժեք ընդունող, բայց արժեք չվերադարձնող ֆունկցիաների դեպքում: Պարզապես այս ֆունկցիան չի պահանջում արգումենտներ, ուստի նրա անունից հետո փակագծերը թողնվում են դատարկ:

Արդյունք՝

```
===== RESTART: C:\User
Hi! Welcome to GenerationAi
>>>|
```

Մուտքային արժեք չընդունող և արժեք վերադարձնող

Այս տեսակի ֆունկցիաները չեն պահանջում մուտքային տվյալներ, սակայն վերադարձնում են արդյունք: Դրանք հաճախ օգտագործվում են ֆիքսված, հաճախ օգտագործվող արժեք վերադարձնելու համար, օրինակ՝ կարգաբերման պարամետրեր, ֆիզիկական հաստատուններ և այլն:

Օրինակ՝

Եկեք ստեղծենք ֆունկցիա, որը վերադարձնում է π (pi) արժեքը՝ բարձր ճշտությամբ, առանց `math` գրադարանի օգտագործման: Այս ֆունկցիան կարող է օգտագործվել հաշվարկներում որպես փոխարինող `math.pi`-ին:

Այստեղ մենք օգտագործում ենք `get_pi` ֆունկցիայի կոդմից տրամադրված π (pi) արժեքը՝ շրջանագծի մակերեսը հաշվարկելու համար:

Ֆունկցիան վերադարձնում է հաստատուն արժեք, որը ֆունկցիայից դուրս օգտագործվում է մակերեսի հաշվարկի մեջ՝ $S = \pi \times r^2$ բանաձևով:

Արդյունք՝

```
===== RESTART: C:\U:
Radius - 5
S - 78.53981633974483
>>>
```

Կարևոր է հիշել, որ ֆունկցիան կարող է ընդունել և վերադարձնել ցանկացած տիպի արժեք՝ թվեր (`int`, `float`), տեքստեր (`str`), ցուցակներ (`list`), բառարաններ (`dict`), տրամաբանական արժեքներ (`bool`), և նույնիսկ այլ ֆունկցիաներ:

Սա դարձնում է ֆունկցիաները շատ ճկուն և հզոր գործիք՝ տարբեր տեսակի տվյալների մշակման համար:

Global և local փոփոխականներ

Ֆունկցիաներ կիրառելիս կարևոր է հիշել, որ՝

- Ֆունկցիայի մուտքում նշված կամ նրա ներսում ստեղծված փոփոխականները կոչվում են տեղային (`local`) փոփոխականներ:

Դրանք գոյություն ունեն միայն ֆունկցիայի ներսում և չեն կարող օգտագործվել ֆունկցիայից դուրս:

- Նույնպես, ֆունկցիայից դուրս հայտարարված փոփոխականները չեն փոխանցվում ավտոմատ կերպով ֆունկցիայի մեջ: Եթե դրանք անհրաժեշտ է օգտագործել ֆունկցիայի ներսում, ապա պետք է հստակ նշել global կամ փոխանցել արգումենտով: Այս առանձնահատկությունը թույլ է տալիս ունենալ նույն անունով փոփոխականներ

Ֆունկցիայի ներսում և դրսում, որոնք միմյանց չեն խանգարում և չեն փոխազդում:
Արդյունք՝

```
===== RESTART: C:\>
x inside function = 5
x outside of function = 10
>>>
```

Այստեղ տեսնում ենք, որ x փոփոխականը, որը հայտարարվել է ֆունկցիայից դուրս, չի փոխվում և ոչ մի կապ չունի ֆունկցիայի ներսում գտնվող x փոփոխականի հետ:

Այսինքն՝ ֆունկցիայի ներսում ստեղծված x-ը բացարձակապես առանձին տեղային (local) փոփոխական է, որը գոյություն ունի միայն տվյալ ֆունկցիայի բլոկում:

Ֆունկցիայից դուրս գտնվող x-ը մնում է անփոփոխ, անկախ այն բանից, թե ֆունկցիայի ներսում նույն անունով ինչ արժեք է տրվում:

Իդեալ, Python-ում մենք ունենք հնարավորություն ծրագրի աշխատանքի ընթացքում տեսնել բոլոր գլոբալ փոփոխականները և դրանց արժեքները:

Սա կատարվում է globals() անունով ներդրված (built-in) ֆունկցիայի օգնությամբ:

Այդ ֆունկցիան վերադարձնում է բառարան (dictionary), որտեղ պարունակվում են բոլոր գլոբալ փոփոխականների անունները որպես բանալի (key), և դրանց համապատասխան արժեքները՝ որպես արժեք (value):

Օրինակ՝

Արդյունք՝

```
===== RESTART: C:\Users\GenAi\Test.py =====
{'__name__': '__main__', '__doc__': None, '__package__': None, '__loader__': <class 'frozen_importlib.BuiltinImporter'>, '__spec__': None, '__annotations__': {}, '__builtins__': <module 'builtins' (built-in)>, '__file__': 'C:\\Users\\Gor Hovsepyan\\Documents\\1.py', 'x': 10, 'name': 'Ani', 'show': <function show at 0x000001E0344172E0>}
```

Կարմիրով նշվածները հենց մեր գլոբալ փոփոխականներն են:

globals() ֆունկցիայի միջոցով հնարավոր է ինչպես ստանալ գլոբալ փոփոխականի արժեքը, այնպես էլ փոխել այն, ճիշտ այնպես, ինչպես աշխատում ենք սովորական բառարանի (dictionary) հետ:

Սա հնարավորություն է տալիս ֆունկցիայի ներսում մուտք գործել կամ փոխել գլոբալ միջավայրի փոփոխականներ՝ առանց դրանք արգումենտով փոխանցելու կամ global բանալի բառ օգտագործելու:

Սակայն շատ կարևոր է զգուշություն ցուցաբերել.

Եթե globals() բառարանից չնշենք որևէ փոփոխական, ապա այն իրականում դուրս է գալիս գլոբալ տիրույթից:

Եվ եթե հետագայում փորձենք օգտագործել այդ փոփոխականը, ծրագիրը կտա սխալ (NameError):

Ստորև բերված է globals()-ի կիրառման օրինակ, երբ գլոբալ և լոկալ փոփոխականների միջև կոնֆլիկտ չի առաջանում:

Արդյունք՝

```

=====
local - 5
global - 101
>>>

```

globals() ֆունկցիան հնարավորություն է տալիս ծրագրի աշխատանքի ընթացքում ստանալ և փոխել գլոբալ փոփոխականների արժեքները՝ աշխատելով որպես բառարան: Սակայն կա նաև ավելի պարզ տարբերակ՝ ուղղակի մուտք գործել գլոբալ փոփոխականներ global հրամանի միջոցով:

Երբ ֆունկցիայի ներսում օգտագործում ենք global հրամանը՝ հետևյալ ձևով՝ global փոփոխականի—անունն ապա տվյալ անունով փոփոխականը համարվում է գլոբալ, և ֆունկցիայի ներսում կատարվող ցանկացած փոփոխություն ազդում է անմիջապես գլոբալ միջավայրի վրա:

Սակայն ուշադրություն.

Եթե փոփոխականը հայտարարել ենք որպես global, ապա ֆունկցիայի ներսում չենք կարող ստեղծել նույն անունով լոկալ փոփոխական, որովհետև այն արդեն համարվում է գլոբալ:

Օրինակ՝

Վերոնշյալ ծրագիրը հաշվում է թե քանի անգամ է կանչվել ֆունկցիան:

Արդյունք՝

```

=====
count = 2
>>>

```

Առաջադրանքներ

1. Գրիր `$ուսկցիա`, որը տպում է «Բարև աշխարհ»:
2. Գրիր `$ուսկցիա`, որը մուտքում ստանում է անուն և տպում է ողջունի հաղորդագրություն:
3. Գրիր `$ուսկցիա`, որը մուտքում ստանում է թիվ և վերադարձնում է դրա քառակուսին:
4. Գրիր `$ուսկցիա`, որը ստանում է երկու թիվ և վերադարձնում դրանց գումարը:
5. Գրիր `$ուսկցիա`, որը ստուգում է՝ արդյոք մուտքային թիվը զույգ է, թե կենտ:
6. Գրիր `$ուսկցիա`, որը ստանում է անուն և տարի, վերադարձնում է Բարև [անուն], դու [տարի] տարեկան ես տեքստը:
7. Գրիր `$ուսկցիա`, որը ստանում է թիվ և վերադարձնում է դրա `$ակտորիալ`:
8. Գրիր `$ուսկցիա`, որը ստանում է ցուցակ և վերադարձնում է նրա միջին արժեքը:
9. Գրիր `$ուսկցիա`, որը վերադարձնում է `n` արժեքը (3.141592...):
10. Գրիր `$ուսկցիա`, որը ստանում է շառավիղ և վերադարձնում է շրջանագծի մակերեսը:
11. Գրիր `$ուսկցիա`, որը ստանում է նախադասություն և վերադարձնում է բառերի քանակը:
12. Գրիր `$ուսկցիա`, որը ստանում է թիվ և վերադարձնում է `True` եթե այն պարզ թիվ է, հակառակ դեպքում՝ `False`:
13. Գրիր `$ուսկցիա`, որը ստանում է ցուցակ և վերադարձնում է նոր ցուցակ, որը պարունակում է միայն դրական թվերը:
14. Գրիր `$ուսկցիա`, որը ստանում է տեքստ և վերադարձնում է նույն տեքստը, բայց գլխատառերը փոքր դարձած:
15. Գրիր `$ուսկցիա`, որը ստանում է $ax^2 + bx + c = 0$ տեսքի հավասարման գործակիցները և վերադարձնում է արմատները ցուցակի տեսքով, բացակայության դեպքում վերադարձնում է դատարկ ցուցակ:
16. Գրիր `$ուսկցիա`, որը օգտագործում է `global` հրամանը՝ `$ուսկցիա`-ի կանչից հետո մեծացնելով գլոբալ փոփոխականի արժեքը 1-ով:
17. Ստեղծիր `$ուսկցիա`, որը `globals()`-ի միջոցով փոխում է գլոբալ `message` փոփոխականի արժեքը նոր հաղորդագրությամբ:
18. Ստեղծիր `$ուսկցիա`, որի ներսում կա նույն անունով լոկալ և գլոբալ փոփոխական, և `globals()`-ի միջոցով փոխվում է միայն գլոբալը:
19. Գրիր `$ուսկցիա`, որը ստուգում է՝ արդյոք `globals()`-ում գոյություն ունի որոշակի անունով փոփոխական, և եթե այո՝ տպում է նրա արժեքը:

20. Գրիր \$ուևկցիա, որը ջնջում է գլոբալ փոփոխական x-ը globals()-ի միջոցով: \$ուևկցիայի կանչից հետո փորձիր տպել x-ը և բացատրիր արդյունքը:
21. Գրիր pow անունով \$ուևկցիա որը մուտքում ստանում է երկու թիվ և վերադարձնում առաջին թվի երկրորդ թիվ աստիճանը:

Թեմա 9. \$այլեր (Files)

Մինչ այժմ մենք բոլոր տվյալները պահպանում էինք փոփոխականների մեջ, և դրանք պահվում էին համակարգչի **օպերատիվ հիշողության** (RAM) մեջ: Սակայն իրական կիրառություններում՝ համակարգչի հետ աշխատելիս, հաճախ անհրաժեշտություն է առաջանում **օգտվել հիմնական (մշտական) հիշողությունից**, որը ներկայանում է տարբեր տեսակի \$այլերով:

Այս \$այլերը կարող են լինել՝

- Տեքստային \$այլեր (.txt)
- Աղյուսակային \$այլեր (.csv, .xlsx)՝ Excel կամ այլ աղյուսակային ծրագրերից
- Նկարներ, ձայնային կամ վիդեո \$այլեր, և այլն

Python ծրագրավորման լեզուն մեզ ընձեռում է լայն հնարավորություններ՝ այս \$այլերի հետ աշխատելու համար՝ ինչպես բացելու, կարդալու և փոփոխելու, այնպես էլ նոր \$այլեր ստեղծելու:

Աշխատանք տեքստային \$այլերի հետ

Համակարգչում \$այլերի հետ աշխատելիս, առաջին քայլը սովորաբար \$այլը բացելն է, որպեսզի հնարավոր լինի կարդալ կամ փոփոխել դրա պարունակությունը: Նույնն է նաև Python ծրագրավորման լեզվում:

Python-ում \$այլ բացելու համար օգտագործվում է open() հրամանը: Սա ամենահիմնական և հզոր գործիքն է \$այլի հետ աշխատանքի համար:

Օրինակ՝

Այստեղ՝

- **file_name** – բացվող \$այլի անունը կամ ուղին

- **mode** – ռեժիմ՝ նշում է, թե ինչպես բացել ֆայլը
- **encoding** (ըստ անհրաժեշտության կարող է չնշվել) – նշում է կոդավորումը (օրինակ 'utf-8' օգտագործվում է հայերեն տեքստը ճանաչելու համար, 'cp1251')
- **errors** (ըստ անհրաժեշտության, կարող է չնշվել սակայն եթե առաջանա կոդավորման սխալ ծրագիրը կդադարեցնի իր աշխատանքը) – նշում է ինչպես վարվել կոդավորման սխալների դեպքում հնարավոր դեպքերը՝

errors արժեք	Վարք
'strict' (լռելայն)	Ցուցադրում է սխալ (Exception)
'ignore'	Անտեսում է սխալ սիմվոլները
'replace'	Փոխարինում է սխալ սիմվոլները ❖-ով
'backslashreplace'	Սխալ սիմվոլները դարձնում է \uXXXX կամ \xxx ձևով
'namereplace'	Օգտագործում է \N{name} ձևով անվանափոխում

Ֆայլերի բացման բոլոր հնարավոր ռեժիմները

Ռեժիմ	Նկարագրություն	Տիպ
'r'	Բացում է միայն կարդալու համար	Տեքստային
'w'	Գրելու ռեժիմ, ջնջում է ֆայլի բովանդակությունը կամ ստեղծում նոր ֆայլ	Տեքստային
'a'	Ավելացնում է ֆայլի վերջում	Տեքստային
'x'	Ստեղծում է նոր ֆայլ, եթե արդեն գոյություն ունի՝ սխալ է տալիս	Տեքստային
'r+'	Կարդալ և գրել (առանց ջնջելու)	Տեքստային
'w+'	Գրել և կարդալ, բայց ջնջում է բովանդակությունը	Տեքստային
'a+'	Ավելացնել և կարդալ	Տեքստային
'rb'	Կարդալ բայթերով (բիթային ռեժիմ)	Բայթային
'wb'	Գրել բայթերով (ջնջում է բովանդակությունը)	Բայթային
'ab'	Ավելացնել բայթերով	Բայթային
'rb+'	Կարդալ և գրել բայթերով	Բայթային
'wb+'	Գրել և կարդալ բայթերով (ջնջում է բովանդակությունը)	Բայթային

Երբ ֆայլ ենք բացում Python-ում open() հրամանի միջոցով, մենք համակարգին տալիս ենք ռեսուրս (հիշողություն, կապ), որը պետք է փակել օգտագործելուց հետո:

- Եթե չփակենք \$այլը, ապա.
\$այլը կարող է մնալ արգելափակված,
- Կրկին բացելու փորձի ժամանակ հնարավոր է ստանանք սխալներ (օրինակ՝ `PermissionError Windows-ում`),
- Ռեսուրսները չեն ազատվում, ինչը խնդիր կդառնա մեծ քանակությամբ \$այլերի աշխատանքի դեպքում:

Նմանատիպ խնդիրներից խուսափելու համար Python-ում օգտագործվում է `with open` կառուցվածքը, որը \$այլը բացելու և օգտագործելուց հետո ավտոմատ կերպով փակում է այն, նույնիսկ եթե ծրագիրը խափանվի կամ սխալ առաջանա:

Սա կոչվում է կոնտեքստ մենեջեր և համարվում է \$այլերի հետ աշխատելու ամենաանվտանգ և խորհուրդ տրվող տարբերակը:

Օրինակ՝

Այստեղ՝

- `file` փոփոխականը պահում է բացված \$այլի օբյեկտը
 - \$այլը ավտոմատ փակվում է, երբ դուրս ենք գալիս `with` բլոկից
- `open()` ֆունկցիայի ռեժիմների համեմատական աղյուսակ

Ռեժիմ	Նպատակ	Եթե \$այլը գոյություն չունի	Եթե \$այլը գոյություն ունի	Կարելի է կարդալ?	Կարելի է գրել?	Օրինակ
'r'	Կարդալ	Սխալ կգեներացնի (<code>FileNotFoundError</code>)	Կկարդա պարունակությունը	Այո	Ոչ	<code>open("file.txt", "r")</code>
'w'	Գրելու (թարմ)	Կստեղծի \$այլը	Կջնջի ամբողջ պարունակությունը	Ոչ	Այո	<code>open("file.txt", "w")</code>
'a'	Գրելու	Կստեղծի \$այլը	Կավելացնի	Ոչ	Այո	<code>open("file.txt",</code>

	(ավելացնել)		վերջում			"a")
'r+'	Կարդալ և գրել	Սխալ կգեներացնի	Կարդալ / փոխել	Այո	Այո	open("file.txt", "r+")
'w+'	Գրելու և կարդալու (թարմ)	Կստեղծի ֆայլը	Կջնջի պարունակությունը	Այո	Այո	open("file.txt", "w+")
'a+'	Կարդալ և ավելացնել	Կստեղծի ֆայլը	Կավելացնի վերջում	Այո	Այո	open("file.txt", "a+")
'rb'	Կարդալ բայթերով	Սխալ կգեներացնի	Կկարդա բայթային ձևով	Այո	Ոչ	open("image.png", "rb")
'wb'	Գրելու բայթերով	Կստեղծի ֆայլը	Կջնջի պարունակությունը	Ոչ	Այո	open("image.png", "wb")
'ab'	Ավելացնել բայթերով	Կստեղծի ֆայլը	Կավելացնի վերջում	Ոչ	Այո	open("image.png", "ab")

Python-ում, ֆայլը բացելուց հետո, մենք կարող ենք կարդալ դրա պարունակությունը երեք տարբեր ձևով՝ կախված այն բանից, թե ինչպես ենք ցանկանում օգտագործել տվյալը: data.txt ֆայլի պարունակություն

1. read() — Կարդում է ֆայլի ամբողջ բովանդակությունը որպես մեկ տող/string

Այս մեթոդը օգտագործվում է այն դեպքում, երբ ցանկանում ենք ամբողջ ֆայլը ստանալ մեկ ամբողջական տողում(string տիպի փոփոխականում):

Արդյունք՝

```

>>>
=====
Բարև աշխարհ
Ես Աբ սերունդից եմ
Սովորում եմ python

```

2. `readline()` — Կարդում է *ֆայլից միայն մեկ տող* (յուրաքանչյուր կանչով հաջորդ տողը)

Արդյունք՝

```
=====
Բարև աշխարհ

Ես Պր սերունդից եմ

>>> |
```

Այստեղ առաջացել է նոր դատարկ տող քանի որ համակարգչի ստանդարտ տեքստային խմբագրիչը նոր տող անցնելու համար օգտագործում է \n որը կարդալիս պահպանվում է անկախ նրանից որ տեքստային խմբագրիչում այն չի երևում:

3. `readlines()` — Կարդում է *ֆայլի բոլոր տողերը* և վերադարձնում է ցուցակ, որի յուրաքանչյուր տարր մեկ տող է

Արդյունք՝

```
===== RESTART: C:\Users\GenAi\Test.py =====
['Բարև աշխարհ \n', 'Ես Պր սերունդից եմ \n', 'Մովորում եմ python']

>>> |
```

Կարդակու բոլոր մեթոդները

Մեթոդ	Վերադարձնում է	Օգտագործվում է երբ...
<code>read()</code>	Տող (string)	Պետք է ամբողջ ֆայլը որպես տեքստ
<code>readline()</code>	Տող (string)	Պետք է հերթով կարդալ տող առ տող
<code>readlines()</code>	Ցուցակ (list of strings)	Պետք է վերամշակել ֆայլի յուրաքանչյուր տող

Բացի կարդալը մենք կարող ենք նաև այդ ֆայլերում գրել: գրելու համար ունենք 2 հրաման սակայն եթե ցանկանում ենք ֆայլում գրել հարկավոր է այն բացել w , a կամ x ռեժիմում:

`write()` — Գրում է տեքստային տվյալ մեկ ամբողջական տողով

Արդյունքում \$այլի պարունակությունը դարձավ՝

Այս դեպքում \$այլի սկզբնական ամբողջ պարունակությունը ջնջվում է, և գրանցվում է նոր տվյալ, քանի որ \$այլը բացվել է 'w' ռեժիմով: Իսկ եթե ցանկանում ենք պահպանել \$այլի նախորդ պարունակությունը, ապա անհրաժեշտ է այն բացել 'a' (append) ռեժիմով՝ ավելացնելու նպատակով:

Ի դեպ, հարկավոր է հիշել, որ եթե \$այլը բացված է միայն կարդալու ռեժիմով ('r'), ապա գրելու փորձը (`write` հրամանը) ծրագրի աշխատանքի ընթացքում կբերի սխալի (`error`):

2. `writelines()` — Գրում է բազմաթիվ տողեր (ցուցակի կամ այլ `iterable`-ի տեսքով)
Արդյունքում \$այլի պարունակությունը դարձավ՝

Այս դեպքում, քանի որ օգտագործվում է 'a' (append) ռեժիմը, ծրագիրը ավելացնում է նոր տողերը \$այլի վերջում: Քանի որ նախորդ անգամ \$այլի մեջ տեղեկատվություն գրելու

Ժամանակ վերջին տողից հետո չկար \n, ապա նոր գրվող տեղեկատվության առաջին տողը միացվել է նախորդի վերջին տողին՝ որպես շարունակություն:

Այս ռեժիմը հատկապես հարմար է, երբ ցանկանում ենք միաժամանակ գրել մեծ քանակությամբ տվյալներ: Ի դեպ, պարտադիր չէ, որ գրվող տվյալը լինի տեքստային. այն կարող է լինել ցանկացած տիպի, օրինակ՝ թվային արժեք:

Ինչպես տեսանք վերոնշյալ աղյուսակում, գոյություն ունեն նաև \$այլ բացելու համակցված ռեժիմներ: Օրինակ՝

- 'r+' – թույլ է տալիս և կարդալ, և գրել, առանց ջնջելու \$այլի բովանդակությունը
- 'w+' – նույնպես թույլ էA տալիս կարդալ և գրել, սակայն բացումից անմիջապես հետո \$այլի ամբողջ պարունակությունը ջնջվում է: Այս ռեժիմով \$այլը բացելուց հետո, եթե փորձենք նախ կարդալ առանց որևէ բան գրելու, ապա կստանանք դատարկ տող (empty string):

Իսկ 'a+' ռեժիմը, ինչպես 'a', օգտագործվում է գրելու և կարդալու համար՝ պահպանելով արդեն գոյություն ունեցող տվյալները և նոր բովանդակությունը ավելացնելով վերջում: Ինչ վերաբերում է բայթային ռեժիմներին ('rb', 'wb', 'ab' և այլն), դրանք աշխատում են նույն կերպ, ինչ իրենց տեքստային տարբերակները, պարզապես \$այլի պարունակությունը կարդում կամ գրում են բայթերով (bytes), ոչ թե Unicode տեքստով: Սա հատկապես կարևոր է, երբ աշխատում եք նկարների, տեսահոլովակների կամ այլ բինար \$այլերի հետ::

Աշխատանք CSV \$այլերի հետ

CSV (Comma-Separated Values) \$այլերը հանդիսանում են տեքստային \$այլեր, որոնցում տվյալները պահվում են աղյուսակային ձևով, որտեղ յուրաքանչյուր տող ներկայացնում է մեկ գրառում (record), իսկ սյունակները բաժանվում են ստորակետով (կամ այլ բաժանարարով, օրինակ՝ ;, \t կամ |):

Օրինակ՝

Այս օրինակում՝

- Առաջին տողը՝ սյունակների վերնագրերն են
- Հաջորդ տողերը՝ յուրաքանչյուր աշակերտի գնահատականներն են երեք առարկաներից և հաշվված միջին բալը

Իդեալ հետագա օրինակներում օգտագործվելու է հենց այս պարունակությամբ data.csv \$այլը: Իհարկե, CSV \$այլերը կարելի է կարդալ որպես սովորական տեքստային \$այլ՝ օգտագործելով open() հրամանը: Սակայն այդ դեպքում ստացված տվյալները կլինեն մեկ ամբողջական տող

կամ տողերի ցանկ, և անհրաժեշտ կլինի ձեռքով մշակել դրանք՝ բաժանելով ըստ ստորակետի, վերածելով ցուցակների և ապա՝ աղյուսակի կառուցվածքի: Այս գործընթացը կարող է բարդ և սխալների հնարավորությամբ լի լինել:

Դրա համար Python-ը առաջարկում է csv ներկառուցված գրադարանը, որը հնարավորություն է տալիս հեշտությամբ կարդալ CSV ֆայլերը որպես աղյուսակային տվյալներ՝ տող առ տող բաժանելով սյունակները:

CSV ֆայլից տվյալների կարդալը csv.reader-ի օգնությամբ:

csv.reader-ը Python-ի csv գրադարանի գործառնություն է, որը թույլ է տալիս CSV (Comma-Separated Values) ֆայլերը հեշտությամբ կարդալ որպես աղյուսակային տվյալներ:

Երբ մենք բացում ենք CSV ֆայլ և փոխանցում այն csv.reader-ին, այդ ֆունկցիան ֆայլի պարունակությունը բաժանում է տողերի այնուհետև Յուրաքանչյուր տող վերածվում է ցանկի (list), որի յուրաքանչյուր տարրը ներկայացնում է համապատասխան սյունակի տվյալը: Օրինակ՝

Արդյունք՝

```
===== RESTART: C:\Users\GenAi\Te
['Անուն', 'Մաթեմաթիկա', 'Ֆիզիկա', 'Քիմիա', 'Միջին բալ']
['Արամ', '85', '78', '92', '85.0']
['Նանա', '90', '88', '95', '91.0']
['Դավիթ', '65', '70', '60', '65.0']
['Լուսինե', '100', '98', '97', '98.3']
>>>
```

Կարևոր նշումներ

Բնութագիր	Նկարագրություն
Տողը որպես ցուցակ	Յուրաքանչյուր տող ստացվում է որպես list, օրինակ՝ ['Արամ', '85', '78']
Տվյալները str են	Բոլոր արժեքները սկզբում ունեն string (տեքստային) տիպ
Վերնագիր (header)	Սովորաբար առաջին տողը պարունակում է սյունակների անունները
Բաժանարարը՝ ,	Սկզբնապես csv.reader օգտագործում է ստորակետ , որպես բաժանարար, սակայն կարելի է փոխել՝ delimiter=";"

Csv ֆայլում ինֆորմացիա գրելը

csv.writer-ը Python-ի csv գրադարանի գործառնություն է, որը թույլ է տալիս տվյալները գրել CSV ֆորմատով ֆայլի մեջ՝ օգտագործելով տողեր և սյունակներ (ինչպես աղյուսակում):

writer-ը աշխատում է հետևյալ հերթականությամբ

1. Սկզբում բացում ենք ֆայլը գրելու ռեժիմով ("w" կամ "a"՝ ավելացնելու համար)
 2. Ստեղծում ենք writer օբյեկտ՝ `csv.writer(file)` հրամանով
 3. Օգտագործում ենք `.writerow()` կամ `.writerows()` մեթոդները՝ տվյալներ գրելու համար
- Օրինակ՝

Արդյունքում ստեղծվել է նոր `students.csv` ֆայլը որը իր մեջ պահում է աշակերտների անունները և գնահատականները:

Կարևոր նշումներ

Փափուկություն	Նկարագրություն
<code>writerow([...])</code>	Գրելու է մեկ տող (list կամ tuple)
<code>writerows([[...], [...]])</code>	Գրելու է միանգամից մի քանի տող
<code>newline=""</code>	Անհրաժեշտ է Windows-ում՝ լրացուցիչ դատարկ տողերից խուսափելու համար
Տվյալները ինքնաշխատ բաժանվում են ստորակետով	Օգտագործիր <code>delimiter=";"</code> եթե բաժանարարը պետք է լինի այլ սիմվոլով օրինակ ;

Արդյունքում ցուցակի պարունակությունը կստեղծվի՝ `students.csv` ֆայլը հետևյալ պարունակությամբ՝

Երբ անհրաժեշտ է գրել մի քանի տող, ավելի հարմար է օգտագործել `writerows()` մեթոդը, որը որպես մուտք ընդունում է ցուցակ, որի յուրաքանչյուր տարրը նույնպես ցուցակ է (այսինքն՝ ցուցակների ցուցակ):

Ինչպես նաև csv գրադարանը ունի հնարավորություն կարդալու և գրելու որպես բառարան՝ օգտագործելով `DictReader` և `DictWriter`, որը մուտքում ստանում է բառարան, որտեղ գրված է սյունակի անունը որպես key և արժեքը որպես value, իսկ կարդալիս վերադարձնում է

բառարան նույն կերպ:

Ստորև բերված է students.csv ֆայլը կարդալու օրինակը:

Օրինակ՝

Արդյունք՝

```
===== RESTART: C:\Users\G
{'Անուն': 'Արամ', 'Մաթեմաթիկա': '85', 'Ֆիզիկա': '78'}
{'Անուն': 'Նանե', 'Մաթեմաթիկա': '90', 'Ֆիզիկա': '88'}
>>> |
```

Ինչպես նաև ստորև կարող եք տեսնել, թե ինչպես գրել ֆայլում՝ օգտագործելով DictWriter-ը:
Արդյունքում ֆայլի պարունակությունը հեռացվում է և գրվում է նորը ստորև կարող եք տեսնել
student.csv ֆայլի պարունակությունը՝

Ինչպես և նախորդ դեպքերում, DictWriter-ի դեպքում նույնպես կարող ենք միանգամից մի քանի տող գրել: Դրա համար օգտագործվում է writerows հրամանը: Ստորև բերված է օրինակը:

Արդյունքում \$այլի պարունակությունը հեռացվում է և գրվում է նորը ստորև կարող եք տեսնել student.csv \$այլի պարունակությունը`

Այս օրինակում օգտագործում ենք a ռեժիմում ավելացնելու համար իսկ եթե ցանկանանք նորը ստեղծել կամ ջնջել և նոր ավելացնել ընդամենը ռեժիմը հարկավոր է փոխել w և ավելացնել writer.writeheader() հրամանը սյուների անունները նշելու համար:

Pandas գրադարան

csv գրադարանը պարզ և հարմար գործիք է CSV \$այլերը կարդալու և ստեղծելու համար, սակայն այն չի կարող կատարել վիճակագրական հաշվարկներ կամ կարդալ .xlsx \$այլեր: Բացի այդ, մեծ քանակությամբ տվյալների հետ աշխատելիս այն բավականին անհարմար է: Վերոնշյալ բոլոր գործողությունները կատարելու համար մեզ օգնության է հասնում pandas գրադարանը:

Pandas-ը Python-ի հզոր գրադարան է, որը նախատեսված է տվյալների վերլուծության և մշակման համար: Այն ապահովում է պարզ և արդյունավետ գործիքներ` աղյուսակային տվյալների հետ աշխատելու նպատակով:

- Այն հնարավորություն է տալիս` DataFrame և Series կառուցվածքներ` տվյալների պատկերման համար:

- Տվյալների ներբեռնում CSV, Excel, SQL և այլ աղբյուրներից:
- Տվյալների մաքրում և վերամշակում՝ բացակայող արժեքների, կրկնօրինակների և սխալների կառավարում:
- Ֆիլտրում, խմբավորում և ամփոփում՝ տվյալների խորքային վերլուծության համար: Վիճակագրական հաշվարկներ:

Սակայն ի տարբերություն csv կամ math գրադարանների՝ pandas-ը չի հանդիսանում Python-ի ներկառուցված գրադարան, և այն օգտագործելու համար հարկավոր է նախ ներբեռնել: Ներբեռնելու համար հարկավոր է մուտք գործել Python-ի տերմինալ (կամ հրամանի տող / Command Line / Anaconda Prompt) և աշխատեցնել հետևյալ հրամանը՝

Այս գործողությունը կատարելուց հետո արդեն կարող ենք ներմուծել գրադարանը և աշխատել նրա հետ:

Աշխատանք ֆայլերի հետ pandas գրադարանի միջոցով

pandas գրադարանը զգալիորեն հեշտացնում է CSV ֆայլերի ընթերցումը, վերլուծությունը և մշակումը՝ համեմատած csv գրադարանի հետ:

`read_csv()` ֆունկցիան հնարավորություն է տալիս հեշտությամբ ներբեռնել CSV ֆայլը որպես DataFrame:

օրինակներում օգտագործվող `students.csv` ֆայլը՝

Այս օրինակում օգտագործվում է `as pd` գրառումը, որպեսզի հետագայում գրադարանի ֆունկցիաները կանչելիս հնարավոր լինի օգտագործել կարճ տարբերակ՝ `pd`, և չլինի անհրաժեշտ յուրաքանչյուր անգամ գրել ամբողջական `pandas` անունը: Այսինքն՝ `pd` դառնում է

pandas գրադարանի հապավում, որը գործնականում համընդհանուր ընդունված է Python-ի համայնքում:

Արդյունքը՝

```
=====
      Անուն  Մաթեմաթիկա  Ֆիզիկա
0      Արամ           85      78
1      Նանա           90      88
2    Դավիթ           65      70
3  Տիգրան           45      66
4    Կարինե           90     100
>>>
```

Ինչպես տեսնում եք՝ մենք կարդացինք ֆայլը DataFrame-ի տեսքով, որը տպելիս ունի ավելի կարգավորված և ընթերցելի տեսք:

Եթե սյունակների անունները չեն ցուցադրվում, ապա կարող եմք ավելացնել դրանք՝ օգտագործելով ստորև բերված օրինակը.

Pandas գրադարանը տալիս է շատ ճկուն հնարավորություններ Excel (.xlsx) ֆայլերի հետ աշխատելու համար, այդ թվում՝ միաժամանակ մի քանի էջերի (worksheet) հետ: Excel ֆայլերից տվյալները կարդալու համար հիմնականում օգտագործվում են read_excel() և ExcelFile մեթոդները:

Excel ֆայլի կարդալը read_excel()-ի օգնությամբ ինչպես և նախորդ օրինակում մենք ստանում ենք DataFrame:

Օրինակներում օգտագործվել է հետևյալ students.xlsx ֆայլը

	A	B	C
1	Name	Python	Ai
2	Tigran	84	76
3	Anush	96	100
4	Vardan	93	100

Էջ 1 անվանումը Charentsavan
անվանումը Vanadzor

Էջ 2

	A	B	C
1	Name	Python	Ai
2	Gor	98	82
3	Anna	87	91
4	Anahit	100	98

Արդյունք

```
=====
      Name  Python  Ai
0      Gor      98  82
1     Anna      87  91
2  Anahit     100  98
>>> |
```

Ինչպես տեսնում ենք այս դեպքում ծրագիրը ավտոմատ կարդաց առաջին էջի պարունակությունը սակայ հնարավորություն կա կարդալու ցանկացած էջը և դա կարող ենք նշել 2 տարբերակով առաջինը անունով երկրորդը ինդեքսով որի հաշվարկը սկսվում է 0-ից

Արդյունք՝

```
=====
      Name  Python  Ai
0  Tigran      84  76
1   Anush      96 100
2  Vardan      93 100
>>> |
```

`read_excel()` հրամանը օգտագործելու դեպքում մենք կարող ենք կարդալ միաժամանակ միայն մեկ էջ կամ բոլոր էջերը՝ որպես առանձին `DataFrame`-ների հավաքածու: Ստորև կարող եք տեսնել օրինակը:

Օրինակ՝
Արդյունքը՝

```
===== RESTART: C:\Users\GenAi\T
{'Charentsavan':      Name  Python  Ai
0      Gor           98    82
1     Anna           87    91
2   Anahit          100   98, 'Vanadzor':      Name  Python  Ai
0   Tigran           84    76
1    Anush           96   100
2   Vardan           93   100}
>>> |
```

Սակայն մենք ունենք հնարավորություն կարդալու բոլոր էջերը միաժամանակ՝ հետագայում դրանք առանձին կարդալու հնարավորությամբ: Դրա համար օգտագործվում է ExcelFile հրամանը, որը հնարավորություն է տալիս նախաբեռնել Excel ֆայլը և հասանելիություն ստանալ բոլոր էջերին, իսկ այնուհետև օգտագործել parse() մեթոդը՝ անհրաժեշտ էջը առանձին կարդալու համար:

Օրինակ՝

Այս օրինակում sheet_names հատկությունը տպում է Excel ֆայլում առկա բոլոր էջերի (էջերի) անունները:
Արդյունք՝

```
=====
['Charentsavan', 'Vanadzor']
      Name  Python  Ai
0      Gor      98  82
1     Anna      87  91
2   Anahit     100  98
>>>
```

parse() հրամանը ոչ միայն հնարավորություն է տալիս առանձնացնել կոնկրետ էջ, այլև նաև հնարավորություն է տալիս ներբեռնել կամ բացառել կոնկրետ սյունակներ կամ տողեր: Ստորև բերված է աղյուսակ, որտեղ ներկայացված են բոլոր հնարավոր պարամետրերը և դրանց օգտագործումը:

Պարամետր	Նկարագրություն	Օրինակ
sheet_name	Ընտրված էջի անունը կամ ինդեքսը	sheet_name="Sheet1"
usecols	Ներբեռնելու համար սյունակների անունները կամ ինդեքսները	usecols="A:C" կամ usecols=[0, 2]
skiprows	Սկզբից բաց թողնվող տողերի քանակը	skiprows=1
nrows	Ներբեռնվող տողերի քանակը	nrows=10
names	Սյունակներին վերագրել սեփական անուններ	names=["Name", "Score"]
index_col	Որ սյունակն օգտագործել որպես ինդեքս	index_col=0
header	Որ տողն օգտագործել որպես վերնագիր	header=0 (առաջին տող), կամ None չօգտագործելու համար
dtype	Սյունակների տեսակների նշում	dtype={"Score": float}

DataFrame-ի ձևափոխման և վերլուծման հրամաններ

head() և tail() մեթոդները pandas գրադարանի շատ օգտակար գործիքներ են, որոնք թույլ են տալիս արագ տեսնել տվյալների սկզբնական կամ վերջնական տողերը՝ առանց ամբողջական աղյուսակը տպելու:

head() – Աղյուսակի առաջին տողերը տեսնելու համար

- Սինտաքս. df.head(n)
- Նշանակություն. Տպում է առաջին n տողերը (լռելյայն՝ 5)

tail() – Աղյուսակի վերջին տողերը տեսնելու համար

- Սինտաքս. df.tail(n)
- Նշանակություն. Տպում է վերջին n տողերը (լռելյայն՝ 5)

Օրինակ՝

Արդյունք՝

```
=====
      Name  Python  Ai
0  Tigran      84   76

      Name  Python  Ai
2  Vardan      93  100
>>>
```

Մեթոդ / Ֆունկցիա	Նկարագրություն	Օրինակ / Արդյունք
df.info()	Տվյալների կառուցվածքի և տիպերի ամփոփում	Name – object, Python – int64, Ai – int64, Տողեր՝ 3
df.describe()	Թվային սյունակների վիճակագրություն	count=3.0, mean(Python)=95.0, mean(Ai)=90.3, min/max=87/100
df.shape	Տողերի և սյունակների քանակ	(3, 3)
df.columns	Սյունակների անուններ	['Name', 'Python', 'Ai']
df.dtypes	Սյունակների տիպեր	Name – object, Python – int64, Ai – int64
df.head(2)	Առաջին 2 տողերը	Gor, 98, 82Anna, 87, 91
df.tail(2)	Վերջին 2 տողերը	Anna, 87, 91Anahit, 100, 98
df.isnull().sum()	Բաց թողնված արժեքների քանակ	Բոլոր սյունակներում 0 բաց թողնված արժեք
df.duplicated().sum()	Կրկնվող տողերի քանակ	0
df['Python'].mean()	Python սյունակի միջին արժեք	95.0
df['Ai'].max()	Ai սյունակի առավելագույն արժեք	98
df['Python'].min()	Python սյունակի նվազագույն արժեք	87
df.corr()	Թվային սյունակների փոխկապակցվածություն	Python ↔ Ai՝ մոտ 0.87

df.value_counts()	Տողերի հաճախականություն	Յուրաքանչյուր տող եզակի է (1 անգամ)
df.nunique()	Յուրաքանչյուր սյունակի տարբեր արժեքների քանակ	Name=3, Python=3, Ai=3

Վերոնշյալները իրենցից ներկայացնում են Pandas գրադարանի հիմնական մեթոդները, որոնք օգտագործվում են DataFrame-ից տվյալներ ստանալու, վերլուծելու կամ առանձնացնելու համար: Այս մեթոդները թույլ են տալիս.

- տեսնել տվյալների կառուցվածքը և տեսակները,
- հաշվարկել վիճակագրական ամփոփագրեր,
- պարզել բաց թողնված կամ կրկնվող արժեքներ,
- ուսումնասիրել սյունակների արժեքների միջակայքները, փոխկապակցվածությունը և հաճախականությունը:

Բացի յոթը նշյալ գործողությունները pandas-ը հնարավորություն է տալիս միավորել աղուսյակները ստորև ներկայացված են աղուսյակները միավորելու հրամանները

Մեթոդ / Յրաման	Նկարագրություն	Օրինակ օգտագործման
pd.concat([df1, df2])	Միավորում է մի քանի DataFrame ուղղահայաց կամ հորիզոնական՝ ըստ ուղղության	pd.concat([df1, df2], axis=0) – տողերով axis=1 – սյունակներով
df1.append(df2)	Ավելացնում է df2-ի տողերը df1-ին (deprecated՝ փոխարինվել է concat-ով)	df1.append(df2)
pd.merge(df1, df2)	Միավորում է երկու աղյուսակ ըստ ընդհանուր սյունակների կամ բանալիների (joins)	pd.merge(df1, df2, on="ID")
df1.merge(df2, how="left")	Merge՝ նշելով ձախ միավորում (left join)	df1.merge(df2, on="ID", how="left")
df1.merge(df2, how="right")	Merge՝ աջ միավորում (right join)	df1.merge(df2, on="ID", how="right")
df1.merge(df2, how="outer")	Արտաքին միավորում՝ ներառում է բոլոր տվյալները երկու աղյուսակներից	df1.merge(df2, on="ID", how="outer")
df1.merge(df2, how="inner")	Ներքին միավորում՝ միայն համընկնող արժեքներով	df1.merge(df2, on="ID", how="inner")
df.join(df2)	Միավորում է df2-ը ըստ ինդեքսների կամ սյունակների	df1.join(df2, lsuffix="_left", rsuffix="_right")
pd.concat([...], keys=...)	Միավորում է և ավելացնում բանալի՝ ենթաշարքեր տարբերվելու համար	pd.concat([df1, df2], keys=['A', 'B'])

Ինչպես նաև մենք կարող ենք պահպանել աղուսյակը և ունենք հնարավորություն պահպանել որպես csv , exel , json ֆայլերի տեսքով ստորև ներկայացված է այդ հրամանների աղուսյակ ինչպես նաև օրինակա թե ինպես ենա միավորում երկու էջերը առանձնացնում միայն անուն 7 python սյունակները և պահում որպես exel ֆայլ:

Հրաման	Նկարագրություն	Օրինակ
<code>df.to_csv("file.csv", index=False)</code>	Պահպանում է տվյալները CSV ֆայլում	<code>df.to_csv("students.csv", index=False)</code>
<code>df.to_excel("file.xlsx", index=False)</code>	Պահպանում է տվյալները Excel ֆայլում	<code>df.to_excel("students.xlsx", index=False)</code>
<code>df.to_json("file.json")</code>	Պահպանում է տվյալները JSON ձևաչափով	<code>df.to_json("students.json")</code>
<code>df.to_html("file.html")</code>	Պահպանում է տվյալները HTML աղյուսակի տեսքով	<code>df.to_html("students.html")</code>
<code>df.to_sql("table", con)</code>	Պահպանում է տվյալները տվյալաշտեմարանում (SQLAlchemy)	<code>df.to_sql("students", con=engine)</code>

Արդյունք՝

```
=====
Վիճակագրական ամփոփում (describe):

              Python
count      6.000000
mean       93.000000
std        6.324555
min        84.000000
25%        88.500000
50%        94.500000
75%        97.500000
max       100.000000

>>>
```

filtered_merge.xlsx ֆայլի պարունակությունը`

	A	B
1	Name	Python
2	Gor	98
3	Anna	87
4	Anahit	100
5	Tigran	84
6	Anush	96
7	Vardan	93

Իդեալ մենք հնարավորություն ունենք վերցնելու դատաֆրեյմի ցանակցած տարր
օգտագործելով loc կամ iloc հրամանները:

Հրաման	Նկարագրություն	Օրինակ (այս տվյալներով)
df.loc[2]	Վերցնում է ինդեքսով 2-րդ տողը (loc-ում` ըստ label)	Վերադարձնում է տվյալ տողի Series
df.iloc[2]	Վերցնում է 3-րդ տողը ըստ ինդեքսի (iloc - integer position)	Վերադարձնում է նույնը, բայց առանց label-հաշվառման
df.loc[1:3]	Վերցնում է տողերը 1-ից մինչև 3-ը ներառյալ	label-ներով ընտրություն
df.iloc[1:3]	Վերցնում է տողերը 1-ից մինչև 3-ը, բացառում է վերջի ինդեքսը	ինչպես Python-ի սլայսերում
df.loc[:, "Name"]	Վերցնում է ամբողջ Name սյունակը	հստակ անունով սյունակի ընտրություն
df.iloc[:, 0]	Վերցնում է 0-րդ (առաջին) սյունակը ըստ ինդեքսի	նույն արդյունքը, բայց ըստ դիրքի
df.loc[0:2, ["Name", "Python"]]	Վերցնում է 0-ից 2 տողերը և միայն նշված սյունակները	բազմակողմանի ընտրություն
df.iloc[0:2, [0, 1]]	Վերցնում է առաջին երկու տողերը և առաջին երկու սյունակները	ըստ ինդեքսների
df.loc[df["Python"] > 90]	Ֆիլտրում է տողերը որտեղ Python > 90	պայմանով տողերի ընտրություն

Օրինակ՝
Արդյունք՝

```
===== RESTART:
Առաջին ուսանողի Python գնահատականը: 98
Python սյունակի միջին արժեքը: 95.0
>>>
```

Առաջադրանքներ

1. Ստեղծիր .txt ֆայլ և գրի այնտեղ երեք նախադասություն՝ օգտագործելով write() մեթոդը:
 2. Բացի վերոնշյալ ֆայլը և տպիր դրա ամբողջ բովանդակությունը տերմինալում՝ օգտագործելով read() մեթոդը:
 3. Կարդա միայն առաջին երկու տողերը ֆայլից՝ readline() մեթոդի օգնությամբ:
 4. Ստեղծիր ցուցակ երեք տողով և գրի այդ ցուցակի պարունակությունը ֆայլում՝ writelines() մեթոդով:
 5. Կարդա բոլոր տողերը ֆայլից որպես ցուցակ՝ օգտագործելով readlines() մեթոդը և տպիր արդյունքը:
 6. Ստեղծիր .csv ֆայլ, որտեղ գրված կլինեն երեք աշակերտի անուն, Մաթեմատիկայի և Ֆիզիկայի գնահատականներ՝ օգտագործելով csv.writer:
 7. Բացի այդ CSV ֆայլը csv.reader մեթոդով և տպիր միայն աշակերտների անունները:
 8. Նույն ֆայլը բացիր csv.DictReader-ով և տպիր միայն այն աշակերտների անունները, ում Մաթեմատիկայի գնահատականը մեծ է 85-ից:
 9. Ավելացրու երկու նոր աշակերտի տվյալներ CSV ֆայլում՝ օգտագործելով csv.DictWriter և writerows() մեթոդը:
 10. Կարդա CSV ֆայլը pandas.read_csv()-ով և տպիր ամբողջ աղյուսակը
- Այսուհետ խնդիրները լուծելու հարկավոր է ստեղծել աղյուսակ հետևյալ պարունակությամբ ֆայլի անուն՝ students_data.xlsx

Sheet 1 – "Grades"

ID	Name	Class	Math	English	Physics
1	Arman	A	85	78	90
2	Narine	B	92	88	95
3	Vardan	A	67	70	60
4	Lusine	B	75	80	85
5	Tigran	A	90	82	88

Sheet 2 – "Attendance"

ID	Name	Days Present	Days Absent
1	Arman	18	2
2	Narine	20	0
3	Vardan	15	5
4	Lusine	19	1
5	Tigran	17	3

Sheet 3 – "Activities"

ID	Name	Sport	Music	Volunteer
1	Arman	Yes	No	Yes
2	Narine	No	Yes	Yes
3	Vardan	Yes	Yes	No
4	Lusine	No	No	Yes
5	Tigran	Yes	No	No

11. Կարդա "Grades" էջը և տպիր այն աշակերտների անունները, որոնց "Math" գնահատականը մեծ է 85-ից:
12. Կարդա "Grades" էջը և հաշվիր Physics սյունակի միջին գնահատականը ամբողջ դասարանի համար:
13. Կարդա "Grades" էջը, ավելացրու նոր սյունակ *Average*, որը հաշվարկում է տվյալ աշակերտի երեք առարկաների միջինը: Պահպանիր թարմացված Excel ֆայլ:
14. Կարդա "Attendance" էջը և գտիր այն աշակերտներին, ովքեր բացակա են եղել 3 կամ ավել օր: Տպիր նրանց անունները:
15. Կարդա "Attendance" էջը, հաշվիր յուրաքանչյուր աշակերտի հաճախման տոկոսը՝ $\frac{\text{Days Present}}{\text{Present} + \text{Absent}} \times 100$ և ավելացրու նոր սյունակ՝ "Attendance %":
16. Կարդա "Activities" էջը և տպիր բոլոր այն աշակերտների անունները, ովքեր մասնակցել են կամ "Sport", կամ "Music" ակտիվությանը:
17. Կարդա "Activities" էջը և հաշվիր քանի աշակերտ մասնակցել է բոլոր երեք ոլորտներին՝ Sport, Music, Volunteer = Yes:
18. Միավորիր "Grades" և "Attendance" էջերը ըստ "ID" կամ "Name", և տպիր այն աշակերտներին, ովքեր ունեն միջին գնահատական < 80 և բացակայել են ավելի քան 2 օր:
19. Միավորիր "Grades" և "Activities" էջերը ըստ "ID", և տպիր նրանց, ովքեր ունեն $\text{Math} \geq 90$ և մասնակցել են Volunteer ծրագրին:
20. Միավորիր "Grades", "Attendance" և "Activities" էջերը ըստ "ID", և պահպանիր նոր Excel ֆայլ՝ որտեղ յուրաքանչյուր աշակերտ ունի իր ամբողջական տվյալները:

Թեմա 10. Խորացված Ֆունկցիաներ (Advanced Functions)

Դիրքային(Positional) և բանալիով (keyword) արգումենտներ

Ֆունկցիաներ թեմայում մենք ծանոթացանք՝

- Ֆունկցիաների տեսակներին,
- դրանց ստեղծման կառուցվածքին,
- լոկալ և գլոբալ փոփոխականների տարբերությանը:

Երբ աշխատում էինք արժեք ընդունող ֆունկցիաների հետ, ֆունկցիայի կանչի ժամանակ մուտքային արժեքները գրում էինք հերթականությամբ, ըստ ֆունկցիայի սահմանման: Այս մեթոդը կոչվում է ֆունկցիայի կանչ՝ դիրքային (positional) արգումենտների միջոցով: Սակայն Python-ում գոյություն ունի նաև մեկ այլ տարբերակ՝ բանալիով արգումենտներ (keyword arguments), որոնց միջոցով կարող ենք հստակ նշել՝ որ արժեքը որ պարամետրին ենք փոխանցում՝ անկախ հերթականությունից: Ստորև կարող եք տեսնել մնամնատիպ կանչի օրինակ՝

Այս օրինակում մենք գրել ենք ֆունկցիա, որը հնարավորություն է տալիս հաշվել շրջանագծի մակերեսը՝ մուտքում ստանալով շառավիղը և π (pi) թիվը:

Դասական, դիրքային արգումենտների (positional arguments) օգտագործման դեպքում ֆունկցիան կանչվում է այսպես՝

`calculate_area(5, 3.14159)` այստեղ առաջին արժեքը (5) համապատասխանում է radius-ին, իսկ երկրորդը (3.14159)՝ pi-ին:

Սակայն keyword arguments-ի դեպքում մենք կարող ենք հստակ նշել, թե որ արժեքը որ պարամետրին է պատկանում՝ անկախ դրանց հերթականությունից:

`calculate_area(pi=3.14159, radius=5)` Այս եղանակը ավելի ընթեմնելի է և թույլ է տալիս փոխել արգումենտների հերթականությունը, քանի որ Python-ը ճանաչում է դրանք ըստ անվան:

Python-ում մենք ունենք հնարավորություն սահմանելու, թե որ պարամետրերը պարտադիր պետք է փոխանցվեն դիրքային (positional) ձևով, իսկ որոնք կարող են փոխանցվել բանալիով (keyword):

Դա իրականացվում է՝ օգտագործելով / նշանը ֆունկցիայի սահմանման մեջ:

Սահմանումը

- /-ից առաջ գտնվող բոլոր պարամետրերը պետք է տրվեն միայն դիրքային ձևով:

- /-ից հետո գտնվող պարամետրերը կարող են տրվել ա՛ դիրքային, ա՛ keyword ձևով:

Օրինակ՝

Արդյունք՝

```
===== RESTART: C:\>
Ani, 25 տարեկան, բնակվում է Gyumri, Armenia
Ani, 25 տարեկան, բնակվում է Gyumri, Armenia
>>> |
```

Այստեղ տեսնում ենք, որ անունը (name) և տարիքը (age) փոխանցվել են որպես դիրքային արգումենտներ, ինչը պարտադիր է, քանի որ դրանք գտնվում են ֆունկցիայի սահմանման մեջ / նշանից առաջ:

Իսկ քաղաքը (city) և երկիրը (country) գտնվում են name և age-ից հետո, այսինքն՝ /-ից հետո, հետևաբար կարող են փոխանցվել ա՛ դիրքային, ա՛ keyword ձևով:

Այս կառուցվածքը թույլ է տալիս վերահսկել՝ ինչ կերպով պետք է ֆունկցիային փոխանցվեն տարբեր պարամետրերը՝ ստիպելով որոշ արժեքներ տրվել միայն դիրքով, իսկ մյուսները՝ ավելի ճկուն ձևով: Եթե փորձենք name և age պարամետրերը, որոնք ֆունկցիայի սահմանման մեջ գտնվում են / նշանից առաջ, փոխանցել որպես keyword arguments, ապա ծրագիրը կվերադարձնի սխալ (TypeError):

Սա պայմանավորված է նրանով, որ / նշանից առաջ գտնվող բոլոր արգումենտները պարտադիր պետք է տրվեն միայն դիրքային ձևով:

Սխալ կանչի օրինակ՝

Արդյունք՝

```
===== RESTART: C:\Users\GenAi\Test.py =====  
Traceback (most recent call last):  
  File "C:\Users\Gor Hovsepyan\Documents\1.py", line 5, in <module>  
    info(name="Ani", age=25, city="Gyumri" , country="Armenia" )  
TypeError: info() got some positional-only arguments passed as keyword arguments: 'name, age'  
>>>
```

Բանալիով (keyword) արգումենտների օգտագործումը հաճախ ծրագրի կոդը դարձնում է ավելի ընթեռնելի և հասկանալի: Դրա շնորհիվ մենք կարող ենք ֆունկցիայի կանչի պահին հստակ նշել, թե որ արժեքն է որ պարամետրին վերաբերող՝ անկախ նրանց դիրքից: Դիրքային և keyword արգումենտների համադրությունն առավել արդյունավետ է հատկապես այն դեպքերում, երբ որոշ արգումենտներ ունեն սկզբնական (default) արժեք: Դա օգտակար է, քանի որ ֆունկցիայի կանչման ժամանակ որոշ արգումենտներ կարող են չփոխանցվել, սակայն ծրագիրը շարունակի աշխատել՝ օգտագործելով նախապես սահմանված արժեքները:

Default արժեք

Սկզբնական արժեք տալու համար, ֆունկցիայի սահմանման մեջ արգումենտի անվան հետևում գրվում է = նշան և համապատասխան արժեքը:

Եթե ֆունկցիայի կանչման ժամանակ այդ արգումենտին արժեք չտրվի, ապա կօգտագործվի այդ սկզբնական արժեքը:

Արդյունք՝

```
=====  
Hello, Anahit:  
Good morning, Gor:  
>>>
```

Կանչի առաջին դեպքում, երբ մենք չենք փոխանցում message պարամետրը, ֆունկցիան օգտագործում է դրա սկզբնական (default) արժեքը՝ օրինակ՝ "Hello":

Իսկ երկրորդ դեպքում, երբ մենք նշում ենք message="Good morning"՝ ֆունկցիան օգտագործում է մեր կողմից տրված արժեքը:

Սա թույլ է տալիս դարձնել ֆունկցիաներն ավելի ճկուն ու վերահսկելի:

Կարևոր է հիշել որ ցանկացած տիպի արգումենտ կարող է ունենալ իր default արժեքը՝ լինի դա թիվ, տեքստ, ցուցակ, բառարան և այլն:

Args kwargs

Վերոնշյալ բոլոր օրինակներում մենք նախապես գիտեինք ֆունկցիայի մուտքային պարամետրերի քանակը: Սակայն երբեմն անհրաժեշտ է լինում, որ ֆունկցիան ընդունի անհայտ քանակությամբ արգումենտներ, և հենց այստեղ է, որ մեզ օգնության է գալիս *args մեխանիզմը:

Ֆունկցիայի ներսում args-ը հանդիսանում է սովորական ցուցակ (list), ինչը նշանակում է, որ կարող ենք դրա վրա կիրառել ցանկացած ցուցակային գործողություն՝ for ցիկլ, len, slicing և այլն

Ստորև բերված է ֆունկցիայի օրինակ, որը ընդունում է անհայտ քանակությամբ պարամետրեր և հաշվարկում է դրանց բոլորի արտադրյալը. Օրինակ՝

Արդյունք՝

```
=====
15
120
1
>>> |
```

Նմանատիպ կերպով, ինչպես *args-ը, Python-ում գոյություն ունի նաև **kwargs մեխանիզմը, որը թույլ է տալիս ֆունկցիային ընդունել անսահման քանակությամբ keyword (անվանված) արգումենտներ:

- kwargs նշանակում է "keyword arguments":

- Այն ֆունկցիայի սահմանման մեջ գրվում է ****kwargs** տեսքով (կամ ******-ով սկսվող ցանկացած անուն):

- Ֆունկցիայի ներսում այն դառնում է բառարան (dictionary), որտեղ
 - բանալիները (keys) – պարամետրերի անուններն են,
 - արժեքները (values) – փոխանցված տվյալները:

Ստորև բերված է ֆունկցիայի օրինակ, որտեղ փոխանցված keyword արգումենտները տավում են՝ ցուցադրելով յուրաքանչյուր պարամետրի անունը և դրա համապատասխան արժեքը:
Օրինակ՝

Արդյունք՝

```
=====
name: Ani
age: 25
city: Charentsavan
age: 25
year: 2000
month: 11
day: 8
>>>
```

Python-ում մենք ունենք հնարավորություն միևնույն ֆունկցիայի սահմանման մեջ միաժամանակ օգտագործելու՝

- սովորական մուտքային արգումենտներ
- ***args**՝ դիրքային անսահմանափակ արգումենտների համար
- ****kwargs**՝ բանալիով անսահմանափակ արգումենտների համար:

Ստորև բերված է այդպիսի ֆունկցիայի օրինակ:
Օրինակ՝

Արդյունք՝

```
===== RESTART:
a = 1
b = 2
args = (3, 4, 5)
kwargs = {'name': 'Ani', 'age': 25}
>>>
```

Բազմակի արգումենտների վերադարձ

Python-ում ոչ միայն հնարավոր է ֆունկցիային փոխանցել անսահման քանակությամբ պարամետրեր, այլ նաև հնարավոր է վերադարձնել մեկից ավելի արժեքներ:

Դրա համար return հրամանի միջոցով ուղղակի նշում ենք բազմաթիվ արժեքներ՝ բաժանելով դրանք ստորակետերով:

Այս արժեքները ավտոմատ փաթեթավորվում են որպես tuple, որը ֆունկցիայի կանչելիս կարող ենք ապափաթեթավորել (unpack)՝ միաժամանակ վերագրելով մի քանի փոփոխականի:

Օրինակ՝

Արդյունք՝

```
>>> |=====
      | 1
      | 9
      | 21
```

Կարճ Ֆունկցիաներ

Python ծրագրավորման լեզուն մեզ հնարավորություն է տալիս գրել ծրագրեր ավելի կրճատ ձևով՝ ընդհանուր ծրագրի ծավալը նվազեցնելու համար: Հենց այդ նպատակով էլ ֆունկցիաներն ունեն կրճատ գրանցման եղանակ: Այդպես գրվող ֆունկցիաները կոչվում են lambda-ֆունկցիաներ:

Սակայն նախքան դրանց ծանոթանալը, անհրաժեշտ է դիտարկել Python-ում առկա այլ կրճատ գրառումներ, օրինակ՝ պայմանական արտահայտությունները և ցիկլերը:

Ցիկլերի կրճատ գրելաձևը հայտնի է որպես ցուցակների համակցում (list comprehension), իսկ ստորև ներկայացված է պայմանների (if) կարճ գրառման տարբերակը:

Պայմանների կրճատ գրելաձև

Պայմանների կրճատ գրելաձևի ժամանակ Python-ում մենք ունենք որոշ սահմանափակումներ:

Առաջինը՝ հնարավոր է գրել միայն `if ... else` կառուցվածքով պայմաններ: Այսինքն՝ անթույլատրելի է օգտագործել միայն `if առանց else`, կամ գրել `if ... elif ... else` կրճատ տարբերակով:

`if ... else`. կրճատ գրելաձևը պահանջում է հատուկ հերթականություն.

- Սկզբում գրում ենք գործողությունը, որը պետք է կատարվի, եթե պայմանի արդյունքը ճիշտ է,
- հետո գրում ենք `if` և պայմանը,
- իսկ հետո `else` և գործողությունը, որը պետք է կատարվի, եթե պայմանի արդյունքը սխալ է

Ստորև ներկայացված է `if ... else` կրճատ գրելաձևի կառուցվածքը Python-ում.

Ստորև բերված է օրինակ, որտեղ `result` փոփոխականում պահվում է "Even" եթե թիվը զույգ է, և "Odd" եթե թիվը կենտ է:

Արդյունք՝

```
=====
Odd
Even
>>>|
```

Ինչպես տեսնում ենք՝ ծրագիրը կատարում է ստուգում և տպում է "Even", եթե թիվը զույգ է, իսկ "Odd"՝ եթե թիվը կենտ է:

Կարճ գրելաձևով այս արտահայտությունը տեղավորվում է ընդամենը մեկ տողի մեջ:

Սովորական պայմանով գրելու դեպքում անհրաժեշտ կլիներ առնվազն 4 տող, օրինակ՝

Lambda ֆունկցիաներ

Այժմ անդրադառնանք lambda-ֆունկցիաներին, որոնք ներկայացնում են ֆունկցիաների կարճ գրանցման ձև:

lambda-ֆունկցիաները նախատեսված են այն դեպքերի համար, երբ անհրաժեշտ է ստեղծել միայն մեկ գործողություն իրականացնող ֆունկցիա, առանց օգտագործելու `def` բանալի բառը: lambda-ֆունկցիա ստեղծելու կառուցվածքը

- Սկսում ենք lambda բառով:
- Այնուհետև գրում ենք մուտքային փոփոխականները՝ ստորակետով բաժանված:
- Դրանից հետո նշում ենք գործողությունը, որի արդյունքը ավտոմատ կերպով վերադարձվում է:

Օրինակ՝

Այս օրինակում `square` անունով փոփոխականը պահում է ֆունկցիա, որը վերադարձնում է փոխանցված թվի քառակուսին:

Ստորև բերված է նաև օրինակ որտեղ մենք տպում ենք մուտքում ստացված երկու փոփոխականները:

Արդյունք՝

```
=====
Sum` 12
>>>|
```

lambda ֆունկցիաները կարող են օգտագործվել նաև պայմանների ստուգման համար:

Օրինակ՝ կարող ենք ստեղծել ֆունկցիա, որը վերադարձնում է True, եթե թիվը դրական է և բաժանվում է 5-ի, հակառակ դեպքում՝ False:

Օրինակ՝

Արդյունք՝

```
=====
True
False
>>>|
```

Այս դեպքում lambda-ֆունկցիայի գործողության հատվածում պարզապես գրվում է բուլյան (Boolean) արտահայտություն, որի արդյունքն էլ ավտոմատ կերպով վերադարձվում է:

Այս ձևը հարմար է հատկապես այն դեպքերում, երբ պետք է ստուգել որևէ պայման և վերադարձնել True կամ False՝ առանց հավելյալ գործողությունների:

Սակայն երբ անհրաժեշտ է վերադարձնել ոչ թե պարզապես True կամ False, այլ տվյալից կախված տարբեր արդյունքներ, օրինակ՝ մոդուլ (բացարձակ արժեք) հաշվելու դեպքում՝ անհրաժեշտ է lambda-ում կիրառել if ... else պայման:

Օրինակ՝ եթե թիվը բացասական է, վերադարձնում ենք այն -1-ով բազմապատկած (այսինքն՝ դրական դարձած), իսկ եթե դրական է կամ զրո՝ վերադարձնում ենք հենց այդ թիվը:

Արդյունք՝

```
=====
7
5
>>>
```

lambda-ֆունկցիաները հիմնականում օգտագործվում են ավելի բարդ աշխատող այլ ֆունկցիաների հետ, ինչպիսիք են map(), filter(), sorted() և այլն:

Այս ֆունկցիաներն իրենց մեջ ընդունում են այլ ֆունկցիա որպես արգումենտ (այսպիսի ֆունկցիաներն անվանում են բարձր կարգի ֆունկցիաներ, կամ higher-order functions), և հենց այստեղ էլ lambda-ն դառնում է շատ հարմար կարճ տարբերակ:

Map ֆունկցիաներ

map() ֆունկցիան նախատեսված է iterable-ի (օրինակ՝ ցուցակի) բոլոր տարրերի հետ նույն գործողությունն ավտոմատ կատարելու համար:

Գործողությունը սահմանվում է որպես ֆունկցիա՝ հաճախ lambda-ի տեսքով, որը փոխանցվում է map()-ի առաջին արգումենտի տեղում:

Երկրորդ արգումենտը հանդիսանում է iterable օբյեկտը (օրինակ՝ ցուցակ), որի տարրերի վրա պետք է կիրառվի գործողությունը:

Ստորև բերված է օրինակ, որտեղ ցուցակի բոլոր տարրերը բարձրացվում են քառակուսի՝ օգտագործելով map() ֆունկցիան:

Օրինակ՝

Արդյունք՝

```
=====
[1, 4, 9, 16, 25]
>>>
```

map() ֆունկցիան պարզեցնում է մեր աշխատանքը՝ կրճատելով կոդի ծավալը և բարձրացնելով ընթերցելիությունը:

Իհարկե, նույն արդյունքը կարելի է ստանալ նաև for ցիկլի միջոցով, սակայն այդ դեպքում ծրագիրը կլինի ավելի ծավալուն և որոշ դեպքերում՝ ավելի բարդ ընթերցելի:

Օրինակ՝

Արդյունքը կրկին կլինի նույնը՝

```
=====
>>> [1, 4, 9, 16, 25]
```

Այսպիսով, բոլոր այն դեպքերում, երբ անհրաժեշտ է կատարել որևէ նույնատիպ գործողություն list-ի, set-ի կամ dict-ի (բառարանի) բոլոր տարրերի վրա, ավելի հարմար և կարճ տարբերակ է օգտագործել հենց map() ֆունկցիան:

Այն օգնում է փոխարինել բազմատող for ցիկլերը պարզ, մեկ տողանոց կոդով՝ առանց կոդի ընթերցելիությունը կորցնելու:

filter ֆունկցիաներ

filter() ֆունկցիան նման է map() ֆունկցիային, սակայն ի տարբերություն map()-ի՝ այն ոչ թե ուղղակի կատարում է բոլոր տարրերի հետ նույն գործողությունը, այլ ստուգում է մեր կողմից տրված պայմանին համապատասխանելիությունը: Եթե համապատասխանում է, ապա այդ տարրը ավելացվում է նոր ստացվող ցուցակում կամ այլ տեսակի հավաքածուում:

filter()-ը նման է իրական կյանքում հանդիպող ջրի ֆիլտրին, որը մաքրում է ջուրը՝ հեռացնելով վնասակար նյութերը: Մեր դեպքում այդ նյութերը որոշվում են ըստ տրված պայմանի:

Գրելաձևը նման է map()-ին. որպես առաջին պարամետր տրվում է ֆունկցիան, որը պետք է վերադարձնի True կամ False, և եթե վերադարձվում է True, ապա տարրը կպահպանվի: Ստորև բերված օրինակում առանձնացնում ենք ցուցակի զույգ տարրերը նոր ցուցակում:

Արդյունք՝

```
=====
[2, 4, 6, 8, 10]
>>>
```

Reduce Ֆունկցիա

reduce()-ը նման է map() և filter() ֆունկցիաներին, սակայն ունի մի կարևոր տարբերություն. այն չի վերադարձնում հավաքածու (list, set...), այլ վերաժում է ամբողջ iterable-ը մի արժեքի: Հասկանալի օրինակ է sum() ֆունկցիան, որը ամբողջ ցուցակի տարրերը գումարում է և վերադարձնում մեկ պատասխան:

reduce()-ը տալիս է նույն հնարավորությունը, պարզապես գործողությունը կարող ենք սահմանել մենք՝ ըստ անհրաժեշտության:

reduce()-ը python-ի ներդրված ֆունկցիա չէ, այն գտնվում է functools գրադարանում և պետք է ներմուծել: Ստորև բերված է օրինակ թե ինչպես հաշվել բոլոր տարրերի արտադրյալը: Օրինակ՝

Արդյունք՝

```
=====
120
>>>
```

Այստեղ reduce()-ը գործարկում է lambda x, y: x * y հետևյալ կերպ՝

1. Սկսում է 1 և 2 $\rightarrow 1 * 2 = 2$
2. Ապա 2 և 3 $\rightarrow 2 * 3 = 6$
3. Ապա 6 և 4 $\rightarrow 6 * 4 = 24$
4. Ապա 24 և 5 $\rightarrow 24 * 5 = 120$

Վերջում ստանում ենք միայն մեկ արժեք:

Իդեալ, նշված բոլոր օրինակներում (map, filter, reduce) մենք օգտագործել ենք lambda ֆունկցիա՝ որպես առաջին պարամետր: Սակայն հնարավոր է օգտագործել նաև սովորական (def-ով ստեղծված) ֆունկցիա, ուղղակի փոխանցելով այդ ֆունկցիայի անունը որպես առաջին արգումենտ:

Ստորև բերված է օրինակ՝ որտեղ օգտագործվում է սովորական ֆունկցիա map()-ի հետ:

Օրինակ՝

Python

```
def square(x):  
    return x ** 2  
  
numbers = [1, 2, 3, 4, 5]  
squared = list(map(square, numbers))  
print(squared)
```

Սովորական ֆունկցիաների օգտագործումը `map()`, `filter()` և `reduce()` ֆունկցիաների հետ մեզ տալիս է հնարավորությունը աշխատելու ոչ միայն պարզ, այլև ավելի բարդ լոգիկա ունեցող գործողությունների հետ:

Եթե գործողությունը չի տեղավորվում մեկ տողի մեջ (ինչպես դա պահանջվում է `lambda`-ի դեպքում), ապա ավելի հարմար է գրել ֆունկցիան `def`-ի միջոցով և այն փոխանցել որպես առաջին պարամետր:

Արդյունք՝

```
=====
```

```
>>> [1, 4, 9, 16, 25]
```

Գրականություն և արտաքին հղումներ

Python 3 documentation: docs.python.org/3/

GeeksforGeeksonline platform <https://www.geeksforgeeks.org/>

W3 school python tutorial: [w3schools.com/python/default.asp](https://www.w3schools.com/python/default.asp)

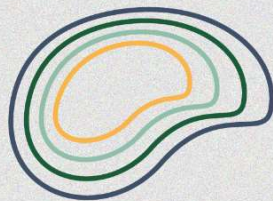
Python in Education: education.python.org/resources/resource/list

Real Python Tutorials: realpython.com

Python documentation -<https://docs.python.org>

Research is Fun: research1.fun/category/python

PYnative Exercises: pynative.com/python-exercises-with-solutions



Generation AI

Nurturing Future Innovators



ՀՀ ԿՐԹՈՒԹՅԱՆ, ԳԻՏՈՒԹՅԱՆ,
ՄՇԱԿՈՒՅԹԻ և ՄՊՈՐՏԻ ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ